



Optimized design refactoring (ODR): a generic framework for automated search-based refactoring to optimize object-oriented software architectures

Tarik Houichime¹ · Younes El Amrani¹

Received: 1 October 2023 / Accepted: 16 May 2024 / Published online: 6 June 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Software design optimization (SDO) demands advanced abstract reasoning to define optimal design components' structure and interactions. Modeling tools such as UML and MERISE, and to a degree, programming languages, are chiefly developed for lucid human-machine design dialogue. For effective automation of SDO, an abstract layer attuned to the machine's computational prowess is crucial, allowing it to harness its swift calculation and inference in determining the best design. This paper contributes an innovative and universal framework for search-based software design refactoring with an emphasis on optimization. The framework accommodates 44% of Fowler's cataloged refactorings. Owing to its adaptable and succinct structure, it integrates effortlessly with diverse optimization heuristics, eliminating the requirement for further adaptation. Distinctively, our framework offers an artifact representation that obviates the necessity for a separate solution representation, this unified dual-purpose representation not only streamlines the optimization process but also facilitates the computation of essential object-oriented metrics. This ensures a robust assessment of the optimized model through the construction of pertinent fitness functions. Moreover, the artifact representation supports parallel optimization processes and demonstrates commendable scalability with design expansion.

Keywords Object oriented design · Search-based refactoring · Complex numbers · Hilbert curve · Object oriented metrics · Genetic algorithms · Particle swarm optimization · Simulated annealing process

✉ Tarik Houichime
tarik_houichime@um5.ac.ma

Younes El Amrani
y.elamrani@um5r.ac.ma

¹ Mohammed V University In Rabat, ENSIAS, Laboratory of Software Project Management, Madinat Al Irfane BP 713, 10112 Agdal-Rabat, Morocco

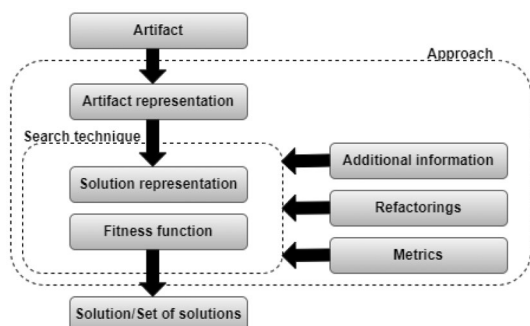
1 Introduction

The software design process is a critical phase in the software development life cycle. It provides a detailed blueprint for the system by specifying the architecture of its components and outlining how they interact with each other. The caliber of software design exerts a considerable impact (Agnihotri and Chug 2022) on the ultimate product's quality, bearing on facets such as functionality, performance, maintainability, and scalability. Hence, optimizing the software design is essential to achieve a high-quality software product. A crucial approach in optimizing software design encompasses the employment of metric-driven methodologies. These metrics (Chidamber and Kemerer 1994) furnish a quantitative foundation for the formulation and verification of software designs, enabling the assessment of various design attributes, such as complexity, cohesion, coupling, and modularity (Agnihotri and Chug 2020). By carefully examining these metrics, software designers can identify potential challenges and opportunities for improvement within the design. This process contributes to enhancing the overall quality and effectiveness of the design.

Search-based refactoring (SBR), as discussed by Mariani et al. in a systematic review of search-based software engineering (Mariani and Vergilio 2017), involves the automated identification of the optimal set of refactorings through the use of optimization heuristics. The application of these refactorings to a particular design leads to an improved software design. This improvement can be assessed and validated through various software metrics, encompassing behavior and execution metrics, as well as metrics related to memory consumption and response time.

The majority of literature studies on SBR adhere to a specific framework and taxonomy for conceptualizing their approaches (Mariani and Vergilio 2017). An overview of this framework is depicted in Fig. 1. Within this framework, the initial input, or artifact, undergoes a transformation into an artifact representation. Additionally, a list of refactorings and metrics are also provided as inputs. The outcome is a solution or multiple solutions that can be applied to the original artifact. In the search technique, it is crucial to establish a solution representation for arranging and assigning the refactorings to specific locations on the

Fig. 1 Overview of search based refactoring approaches (Mariani and Vergilio 2017)



artifact representation where they can be implemented. The search technique subsequently fine-tunes and optimizes this solution representation. Finally, a fitness function evaluates the impact of applying a particular set of refactorings to help steering the optimization process.

Notably, in the realm of optimization problems, the configuration of the solution representation significantly impacts the search technique, as highlighted by Leyman et al. (2019). Specifically, in SBR, the solution representation plays a key role in checking the feasibility of refactorings and assessing them through their application to the artifact representation to collect metrics. Consequently, the efficiency of the employed search technique in SBR also depends on the amount of time spent on validating and evaluating the proposed solutions. Thus, modeling SBR approaches requires careful consideration of how solution representations are encoded and used.

While computer systems are incredibly fast and efficient when it comes to simple and straight-line calculations, turning complex problems into simple calculations isn't straightforward. It requires a clever way to simplify the problem, which means breaking it down into numbers or simple formats like vectors and matrices that computers can work with quickly. This simplification enables computers to leverage their inherent speed in calculations to effectively address the problem at hand, achieving independence between computation and interpretation (Amal et al. 2014; Rapaport 2023; Douglas and Gödel 1999). Simultaneously, it is crucial to encode the minimal information necessary for decoding the final results, thereby enabling accurate inferences and valid conclusions to be drawn from said outcomes. Considering that the optimization heuristics used in SBR are not explicitly designed for challenges specific to SBR, and recognizing that the encoding of a problem for heuristic optimization significantly influences the effectiveness of the optimization process (Li et al. 2017; Schotter et al. 2012; Costa et al. 2013), it is crucial to reformulate these SBR problems into a framework amenable to existing optimization techniques. Particularly, adopting intricate encodings for artifact and solution representations—such as abstract syntax trees (AST) (Sun et al. 2023) and other tree structures, whose size and depth, and thus computational complexity, correlate directly with the encoded artifact's length and dependencies—can significantly impair the heuristic's efficiency and processing speed.

Moreover, the implementation of an independent solution representation component tasked with continuously validating and evaluating solutions against the artifact representation component at each step introduces substantial delays. This iterative verification process, which does not contribute directly to the manipulation and optimization of the solution itself, leads to avoidable delays, thus creating a bottleneck in the heuristic optimization process. Separating the components that represent the artifact and the solutions not only increases the time required for processing but also adds significant extra costs, reducing the effectiveness of the heuristic approach.

Consequently, merging the solution representation and the artifact representation into a unified entity, which serves as both simultaneously, may provide several benefits to the SBR approach. By integrating these representations, the heuristic can directly access the required metrics from the unified entity being optimized, eliminating the need to query these metrics from outside the optimization process which can negatively impact the optimization process. Furthermore, unlike SBR methods

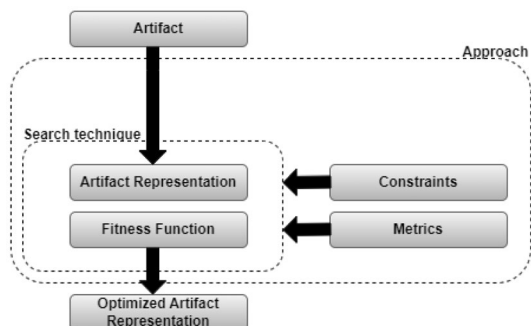
(Ghannem et al. 2013; Kebir et al. 2017) that depend on refactoring sequences (comprising multiple refactorings) as their initial solution representation, offering the flexibility to incrementally adjust the unified representation through minor modifications at each step gives enhanced control over the optimization process (Bringmann et al. 2021; Lyu et al. 2022). This approach also opens up opportunities to address additional challenges, such as refactoring schedule conflict (Hemati Moghadam et al. (2015; Liu et al. 2008) and class assignment problem (Bowman et al. 2010; Masoud and Jalili 2014). Additionally, simplifying the encoding of this unified component could facilitate the use of various existing heuristics with minimal adaptation for the SBR problem.

1.1 Research objectives and approach overview

Inspired by the way mathematicians skillfully utilize abstract equations, such as $Ax = B$, to describe optimization problems by meticulously finding the optimal value of x that minimizes or maximizes the quantity Ax , all while navigating through constraints represented by matrix A , we introduce Optimized Design Refactoring (ODR) framework. This abstraction approach not only underscores the significance of simplifying intricate real-world phenomena into manageable equations but also reflects the cognitive process of sensory gating (Cromwell et al. 2008), wherein essential data is isolated from the noise of extraneous information. ODR seeks to emulate this elegance and efficiency, aiming to abstract the essence of the SBR optimization problems while addressing the inherent challenges of encoding and decoding processes. Through this, we aspire to achieve a balance between the depth of problem encoding and computational efficiency.

By exploiting the inherent algebraic characteristics of complex numbers that provide a method for reducing dimensionality as they integrate both real and imaginary components into a single entity, the complex geometry which possesses inherent attributes that facilitate the manipulation of complex numbers, and the reverse mapping of Hilbert filling curve which enables the encoding of a substantial volume of information within a one-dimensional space (Zhou et al. 2021); ODR offers an innovative framework that is aimed at formulating the problem of SBR in a format that, as we hypothesize in this introductory section, holds the potential to unify the

Fig. 2 Overview of our simplified approach for search-based refactoring



artifact and solution representations into a single unified representation (see Fig. 2) without using computationally demanding structures. Through this unified representation, we hypothesize that the search-based algorithm can directly modify the artifact representation without the need for a separate solution representation. Consequently, the optimization algorithm can directly evaluate and validate the design with minimal decoding requirements, effectively transforming the solution representation into a new improved artifact representation. Notably, the proposed format exhibits sufficient generality to seamlessly integrate with a wide range of search-based optimization algorithms while minimizing the computational resources required for manipulating the problem's components. Such an approach, we argue, renders the problem's modeling largely independent from the employed heuristic.

Additionally, the proposed framework inherently lends itself to parallelization by leveraging matrix computations to generate novel solutions. Also, we employ a simple and efficient technique for representing the various constituents of the problem, ensuring that each component operates independently of others. Particularly in the realm of parallel execution, individual computational units can possess their own distinct vectors that effectively encapsulate the entirety of the artifact, while maintaining independent manipulability and minimizing memory footprint.

In the discourse of this manuscript, we elucidate several novel contributions, enumerated as follows:

- A novel framework for search-based software design refactoring, notable for its adaptability and flexibility. Crucially, the framework is designed to decouple the conceptualization of the optimization problem from the specific heuristic or optimization algorithm employed.
- The introduction of a methodological approach that markedly simplifies the architectural design of search-based refactoring processes. This is achieved by amalgamating the representations of both the artifact and solution, without using computationally demanding structures
- The proposition of a framework model intrinsically equipped to facilitate a substantial array of intuitive refactorings and enact fine-grained modifications to the design, coupled with efficient metric computations
- The presentation of a framework model inherently designed for adaptability, rendering it conducive for parallelization and thereby ensuring its efficacious deployment within parallel computational environments.

1.2 Synopsis

In the upcoming section, we provide a detailed overview along with a contextual framing of the foundational techniques for this study. Following this, we present a comprehensive description of the proposed framework, including its mathematical foundations. Subsequently, the framework is applied to encode a practical use case, explicitly demonstrating how an actual artifact is represented within the framework. The research evaluation methodology is then systematically outlined, covering the articulation of research questions and the discussion of validation mechanisms for

the proposed methodology. This discussion is further enriched by a rigorous analysis that addresses the strengths, limitations, and critiques of the framework itself, as well as its application in refactoring the architecture of a simplified design of an e-commerce management system. The objective is to comprehensively address the set of research questions. Finally, we conclude and suggest topics for future investigations.

2 Foundational concepts

2.1 Hilbert space-filling curve

Since its discovery in the late 19th century, the study of space-filling curves, like the Peano and Hilbert curves, has fascinated researchers. These curves map a one-dimensional line onto a two-dimensional square in a unique and continuous way (Nguyen and Franco 2012). The Hilbert curve, in particular, has a recursive design that makes it valuable for data science. The curve breaks a square into quadrants and keeps subdividing them in a symmetrical pattern. This methodical layout is useful for reducing data dimensions and encoding complex data patterns (Anders 2009) in a simpler form. In essence, the Hilbert curve maintains the relationships between data points while simplifying their representation. Our study leverages the reverse Hilbert space-filling-curve technique for a specific application: encoding attributes of class methods in software, by using the curve's properties, we can efficiently represent complex information as single numerical values. This aids in better and simpler data analysis and interpretation.

2.2 Optimization heuristics

In the realm of computational problem-solving, the application of heuristics for optimization has become a cornerstone, particularly for intricate problems that resist straightforward analytical solutions. Such heuristic methods serve as exploratory tools, neatly navigating the problem space to furnish approximate yet highly useful solutions. While there are myriad heuristics available, a trio of them, which are employed in this study, —genetic algorithms (GA) (Katoch et al. 2021), particle swarm optimization (PSO) (Imran et al. 2013), and simulated annealing (SA) (Rutenbar 1989)—has garnered widespread attention for their robustness and applicability across a broad array of domains.

Genetic algorithms are informed by the mechanisms of biological evolution, particularly natural selection. These algorithms generate and maintain a population of candidate solutions, which undergo processes akin to genetic crossover and mutation (Immanuel and Chakraborty 2019). As generations progress, weaker solutions are typically phased out in favor of stronger, more fit solutions. The success of GA is often attributed to its ability to balance both exploration and exploitation in the

search space, thereby making it conducive for problems where the landscape is fraught with local minima.

On another front, particle swarm optimization presents a contrasting approach based on social interactions among agents in a swarm (Gad 2022). Inspired by the collective behavior witnessed in flocks of birds or schools of fish, PSO operates by having individual particles adjust their position in the problem space based on the best-known positions of themselves and their neighbors. The particle movement is often influenced by factors like cognitive and social components, engendering a cooperative search strategy. The strength of PSO lies in its simplicity and its ability to converge rapidly, making it an excellent choice for continuous optimization problems.

Lastly, simulated annealing takes its conceptual roots from the physical annealing process employed in metallurgy (van Ginneken 2016). The algorithm embarks on a random walk through the solution space, accepting new states not just based on improvements in objective function value, but also probabilistically accepting worse states. This is orchestrated through a temperature parameter that decreases over time, thereby reducing the likelihood of accepting inferior solutions as the algorithm matures. This unique ability to escape local minima makes Simulated Annealing particularly useful for optimization problems characterized by a rugged and discontinuous landscape.

3 Unifying artifact and solution representations

3.1 Classes and properties modeling

The aim of our transformation is to represent a design architecture by means of a unified vector consisting of complex numbers, which can be efficiently and swiftly manipulated through basic algebraic operations. Within this representation, a class property, whether it is an attribute or a class method, is expressed using a single complex number. However, it is important to note that the complex number representing a class property must consider the unique identity of the class to which the property belongs.

To achieve this, the identity of each class within the classes space is encoded by a unique angle denoted as φ , while each property is encoded by a distinct angle denoted as θ . Both the φ and θ angles are then encompassed within the complex number that represents the entirety of the property. This encoding methodology facilitates the ease of property manipulation, as it necessitates only a single algebraic operation.

Let ζ be the classes space, and $\varphi \in \left[-\frac{\pi}{2}, +\frac{\pi}{2}\right]$, $\varphi \neq 0$, we attribute to each class $\eta \in \zeta$ an angle φ such as:

$$\forall \eta \in \zeta \quad \exists! \varphi \in \left[-\frac{\pi}{2}, +\frac{\pi}{2}\right] \implies f_1(\eta) = \varphi$$

where f_1 is a one-to-one application that maps each class from the space ζ to a unique angle φ . We denote as ρ a property pertaining to a specific class which is denoted as η . The set of properties belonging to class η is denoted as Ω , which encompasses all properties that are attributed to η .

Let $\theta \in]0, \pi]$, we attribute to each property ρ a unique angle θ such as :

$$\forall \eta \in \zeta \quad \forall \rho \in \Omega_\eta \quad \exists! \theta \in]0, \pi] \quad \implies f_2(\rho) = \theta$$

where f_2 is a one-to-one application that maps each property to a unique angle θ .

From a geometric perspective, the representation of a class property ρ is achieved by employing a complex number z in the Euler form Gungor (2019), which is characterized by its magnitude r and argument θ as follows:

$$z = r.e^{i\theta} = r.(\cos(\theta) + i \sin(\theta)) \quad (1)$$

To incorporate the class identity into the encoding process (Fig. 3), we extend the definition of the real part of the complex number z to be defined with two polar coordinates z_1 and z_2 , while keeping the imaginary part defined by only one coordinate z_3 as described by Eq. 2. Thus, a complex number can be described by three polar coordinates r_φ , φ , and θ , thereby enabling its representation on a three-dimensional sphere (refer to Figs. 4 and 5).

$$\begin{aligned} z_1 &= r \sin(\theta) \cos(\varphi), \\ z_2 &= r \sin(\theta) \sin(\varphi), \\ z_3 &= r \cos(\theta) \end{aligned} \quad (2)$$

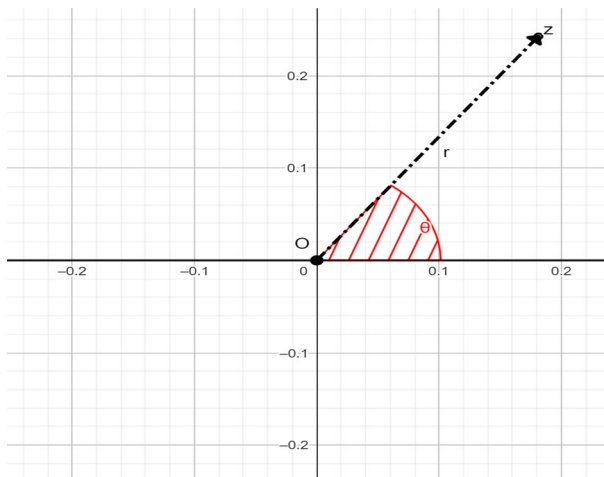


Fig. 3 Two-dimensional visual depiction illustrating the complex number z , which encapsulates a distinct class property ρ , the complex number is represented utilizing its polar coordinates r and θ

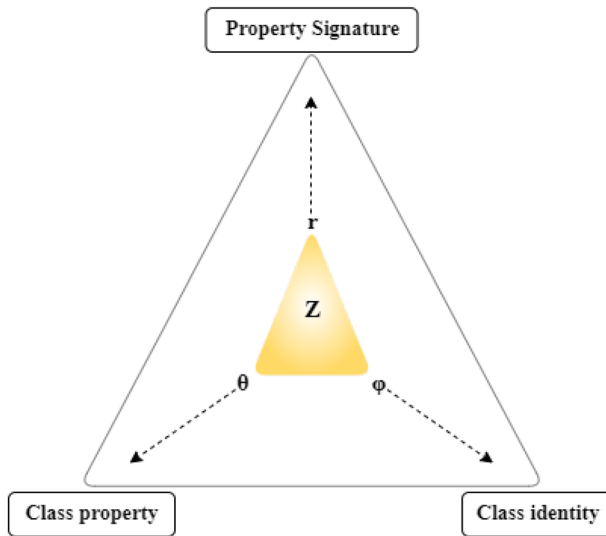


Fig. 4 Mapping the components φ , θ , and r of the complex number Z to object oriented concepts

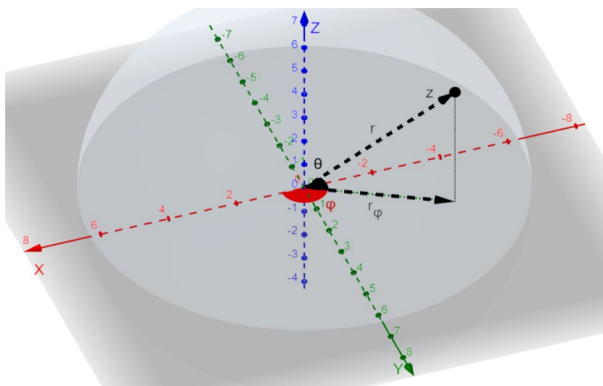


Fig. 5 Three-dimensional visualization depicting the encoding of a class property and its associated class. The class identity is encoded in the angle φ , while the property's identity is encoded in the angle θ

In this novel extension, the properties within the same class are depicted in the plan determined by the vectors r and r_φ (refer to Figs. 5 and 6). Consequently, the arrangement ensures that a change in the angle φ corresponds to a transition between different classes, as the plane formed by these vectors rotates around the vertical imaginary axis (OZ).

It is important to note that the angles φ and θ possess distinct characteristics as they are unique through the global context of the representation. thus, modifying the angle φ causes the rotation of the plane defined by the vectors r and r_φ around the (OZ) axis, but it preserves the class properties associated with that plane, as determined by the angles θ . Moreover, since the angle θ holds its distinctiveness in

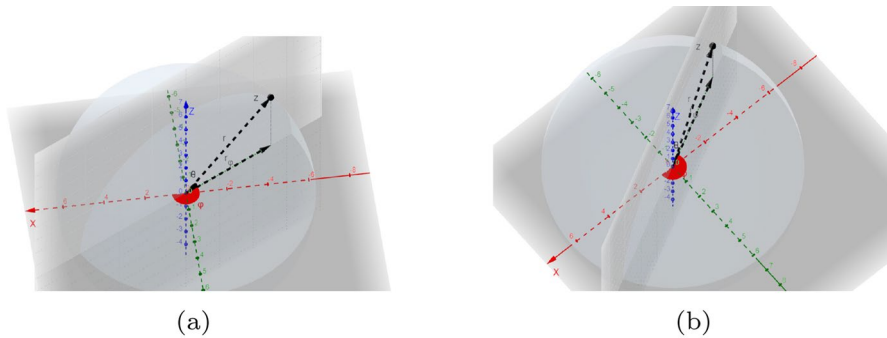


Fig. 6 Three-dimensional visualization from perspectives (a) and (b), illustrating a plan encompassing one or more class properties, each defined by its angle θ . The illustrated plan encompasses all properties belonging to the same class

the global context of the representation, the rotation of the plan that holds the class property encoded by θ does not compromise its uniqueness.

3.1.1 Encoding class properties and their signatures

In the context of a specific class η , a property ρ can be either an attribute or a class method. To encode a class property, the utilization of a complex number $z = r.e^{i\gamma}$ is employed, where γ is reserved to denote the identity of the property. By employing the magnitude r , we can encode the type of the property, indicating whether it is a class attribute or a method, as well as the attributes utilized by the property if it is a class method.

When ρ is an attribute, the magnitude r in the complex number representation, $z = r.e^{i\gamma}$, is set to 1. Consequently, the property is solely represented by its angle γ :

$$z = e^{i\gamma} \quad (3)$$

If we consider the scenario where ρ represents a class method, the magnitude r is assigned a value distinct from unity. This non-unit magnitude is employed to encode the arguments that the class method accepts. These arguments can encompass a wide range of data types, such as user data or attributes derived from either the same class or other classes within the same architectural framework. In the context of the present study, our attention is specifically directed towards arguments that are related to attributes of either the same class or other classes within the given design.

To encode these arguments in the magnitude r , we express the arguments that are passed to the methods of the design as an ordered subset of the set Ω . Formally, we denote this subset as $\lambda = \{\rho_1, \rho_2, \dots, \rho_N\} \in \mathcal{P}(\Omega)$, where $\mathcal{P}(\Omega)$ signifies the power set of Ω . Within this subset, all the attributes utilized in each class method of the design are included, following a predetermined specific order. Throughout the optimization process, the cardinality of the subset, denoted as N ,

remains constant, as our refactoring does not modify the arguments passed to the class methods. Simultaneously, the defined order is maintained. Additionally, we introduce a binary vector $B_\theta = \langle b_1, b_2, \dots, b_N \rangle$ where $b_i \in \{0, 1\}$. This binary vector will serve to encode the utilization of those arguments by a specific method. To determine the elements of the vector B_θ , which are specific to each class method defined by θ , we define the following mapping from λ , where $\lambda \neq \emptyset$, to 0, 1, and this mapping is defined for each class method as follows:

$$b_i = g_\theta(\rho_i) = \begin{cases} 1, & \text{if } \rho_i \text{ is used by method } \theta, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Finally, the resulting vector B_θ for each class method is transformed into a single floating-point number, which represents the magnitude r of the class method. This transformation involves a one-to-one mapping using the Hilbert filling curve, which takes an N -dimensional binary vector as input and produces a one-dimensional distance point along a Hilbert curve as an integer value. Here, N represents also the number of dimensions of the Hilbert curve hypercube ($N > 0$), and I denotes the number of iterations used in constructing the Hilbert curve ($I > 0$). In cases where a class method takes no arguments, a special B_θ vector is obtained, where all b_i are set to 0 s and only b_N (the last position of the vector) is set to 1. Subsequently, we manipulate the obtained integer value by taking the N -th root, yielding the final magnitude calculation as follows:

$$r = \sqrt[N]{H_{(I,N)}(B_\theta)} = \sqrt[N]{H_{(I,N)}(\langle b_1, b_2, \dots, b_N \rangle)} \quad (5)$$

Consequently, we express the complex number that encodes a class method and its signature as follows:

$$z = \sqrt[N]{H_{(I,N)}(B_\theta)} \cdot e^{i\gamma} \quad (6)$$

In situations where the design includes class methods that utilize other class methods, we can extend the binary vector B_θ to include these additional class methods identified by their θ s. It is crucial to incorporate these class methods into the binary vector while adhering to a specific order. By doing so, we ensure that the binary vector accurately represents the utilization of all class methods and class attributes in a consistent manner.

Table 1 Classification of (ρ_i, ρ_j) based on φ and θ

(ρ_i, ρ_j)	$\varphi < 0$	$\varphi > 0$
$\theta_j = \theta_i$	Interface	Superclass
$\theta_j \neq \theta_i$	–	Other classes

3.1.2 Encoding superclasses and interfaces of the design

In software design, a system often consists of classes that represent interfaces and superclasses. In our representation, interfaces and abstract properties are assigned a negative value of φ ($\varphi < 0$), while superclasses have a positive value of the angle φ .

However, when a method ρ_i implements or overrides another method ρ_j , both methods have the same angle θ . This can be expressed mathematically as $f_2(\rho_j) = f_2(\rho_i) \implies \theta_j = \theta_i$. Table 1 summarizes the encoding of these special classes using angles θ and φ . To encode an inheritance chain of any length, we record the identities of the superclasses chain starting from the first superclass (root class) following this constraint: $\varphi_i > \varphi_{i-1} > \dots > \varphi_0 > 0$, where φ_i is the root class (the first class in the chain), and φ_0 is the last subclass or child in the chain.

3.2 Encoding the entire design architecture

To encode the entire design architecture, we construct a column matrix that includes all the attributes and methods present in the architecture. This matrix consists of complex numbers representing the properties of the design. In order to encode the class identities, which are represented by the angle φ , we set $\gamma = \varphi \cdot \theta$, where φ represents the integer part of γ and θ represents the decimal part. Let P denote the total number of class properties (attributes and methods), and C denote the number of classes in the design architecture. Thus, θ and φ can be expressed as follows:

$$d = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_P \end{bmatrix} = \begin{bmatrix} r \cdot e^{i\gamma_1} \\ r \cdot e^{i\gamma_2} \\ \vdots \\ r \cdot e^{i\gamma_P} \end{bmatrix} = \begin{bmatrix} r \cdot e^{i\varphi_1 \cdot \theta_1} \\ r \cdot e^{i\varphi_2 \cdot \theta_2} \\ \vdots \\ r \cdot e^{i\varphi_C \cdot \theta_P} \end{bmatrix} \quad (7)$$

The design matrix, denoted as d , can be expressed in a simplified form:

$$d = \begin{bmatrix} r \cdot e^{iq_1 \cdot k_1} \\ r \cdot e^{iq_2 \cdot k_2} \\ \vdots \\ r \cdot e^{iq_C \cdot k_P} \end{bmatrix} \quad (8)$$

where :

$$\theta_i = k_i \cdot \frac{2\pi}{P}, \quad \varphi_i = q_i \cdot \frac{2\pi}{C}, \quad q_i \in \mathbb{Z}^* \text{ and } k_i \in \mathbb{N}^* \quad (9)$$

Indeed, the vector d combined with the ordered subset λ effectively encodes all the properties of the classes within the design. It provides a comprehensive representation that encompasses the class identities, the number and identities of arguments and other methods utilized by each class method, and finally, the identities of each argument and method in the design.

3.3 Design manipulation for refactoring

The main objective of our study is to encode the design in a representation that preserves its most relevant structural properties, even after refactoring the design, we aim to achieve this in a way that the representation does not require a specific format for manipulation, allowing for flexible, easy and fine-grained modifications. To accomplish this, we have previously encoded the properties of the design within complex numbers. The complex numbers serve as a means to represent the properties and their relationships in a compact and expressive manner.

In the encoding scheme, the rotation of a complex number represents the transfer of a property from one class to another. This rotation operation involves adding or subtracting the arguments of two complex numbers. It is important to note that the argument ω is an integer value ($\omega_i \in \mathbb{Z}$), which means that **only the integer part of the argument of the rotated complex number is affected by the rotation**. The decimal part of the argument remains unchanged during the rotation process. This property ensures that the encoding maintains the identities of the properties and thus, allows for precise tracking of property transfers between classes.

$$\begin{bmatrix} r.e^{iq_1.k_1} \\ r.e^{iq_2.k_2} \\ \vdots \\ r.e^{iq_C.k_P} \end{bmatrix} \begin{bmatrix} e^{i\omega_1.0} & & \\ & e^{i\omega_2.0} & \\ & & \ddots \\ & & & e^{i\omega_C.0} \end{bmatrix} = \begin{bmatrix} r.e^{i(\omega_1.0+q_1.k_1)} \\ r.e^{i(\omega_2.0+q_2.k_2)} \\ \vdots \\ r.e^{i(\omega_C.0+q_C.k_P)} \end{bmatrix} \quad (10)$$

Finally, refactoring the design involves performing a transformation on the design vector using a transformation vector T . The elements of T represent the quantities by which each property in the design should be rotated to achieve the optimal intended design. Therefore, the entire optimization problem revolves around finding the best values for the transformation vector T . By adjusting the values in T , we can effectively control the rotation of each property encoded as a complex number, leading to a systematic refactoring of the design. Thus, any optimization algorithm is supposed to find the most suitable values for T that result in the desired changes to the design while maintaining its initial format and adhering to the defined objectives and constraints.

$$T = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \vdots \\ \omega_C \end{bmatrix} \quad (11)$$

In summary, we enable the manipulation of the design properties through mathematical operations, such as rotations, without the need for a design-specific solution representation format. This approach facilitates the process of refactoring the design while maintaining the integrity and structure of the original design representation.

3.4 Calculating design metrics for fitness function

To align with our objective of providing an efficient way to evaluate and validate the refactored design, we propose a method for calculating a subset of design metrics. As part of this method, we represent the arguments γ of the complex number as a one-dimensional array (see Table 2), where each index corresponds to the decimal part of the argument. This array represents the ordered set $\lambda \cup (\Omega \setminus \lambda)$, which encompasses all the properties of the design, the values stored in the array are the integer parts of the complex numbers. These integer values denote the class identities to which each property belongs.

Subsequently, to determine the indexes of the properties used in each class method identified by θ , we reverse the magnitude r of the complex number. This reversal yields the binary vector B_θ which acts as a filter, indicating the usage of attributes and methods from the subset λ in the corresponding class method. By calculating the Hadamard product of vector B_θ with λ' , we can easily identify the identities of the attributes and the methods from other classes, that are utilized by each class method. The resulting binary vector, denoted as $B_\theta \odot \lambda'$, provides information on the presence or absence of other properties in each class method, facilitating further analysis and evaluation.

$$B_\theta = H_{(I,N)}^{-1}(r^N) \quad (12)$$

where: $H_{(I,N)}^{-1}$ is the reverse function of the Hilbert filling curve.

$$B_\theta \odot \lambda' = \langle b_1, b_2, \dots, b_N \rangle \odot \langle \theta_1, \theta_2, \dots, \theta_N \rangle \quad (13)$$

where:

$$\lambda' = \langle f_2(\rho_1), f_2(\rho_2), \dots, f_2(\rho_N) \rangle = \langle \theta_1, \theta_2, \dots, \theta_N \rangle \quad (14)$$

Thus:

$$B_\theta \odot \lambda' = \langle b_1.\theta_1, b_2.\theta_2, \dots, b_N.\theta_N \rangle \quad (15)$$

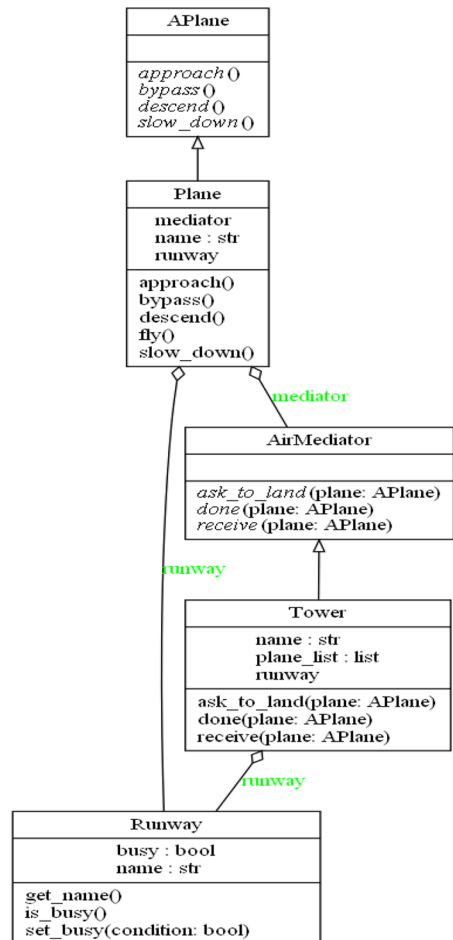
by using this linear encoding scheme, we can directly infer a number of object-oriented metrics like depth of inheritance tree (DIT), coupling between objects (CBO), lack of cohesion (LCOM), and specially those related to classes dependencies, this results in quicker evaluation and validation of refactored designs. In the use case section, we give concrete samples of calculations of these metrics.

4 Use case study: the mediator design pattern architecture

To clarify the transformation methodology within the suggested computational framework, we advocate for the conversion of an object-oriented design aligned with the mediator design pattern (Fig. 7) into a complex vector. In the initial

Table 2 Linear representation of the complex number's arguments, enabling straightforward calculation of an important set of OO metrics

θ_1	...	θ_N	...	θ_P
φ_{θ_1}	...	φ_{θ_N}	...	φ_{θ_P}

Fig. 7 UML diagram of an object-oriented design aligned with the mediator design pattern

phase, angular coordinates are allocated to individual classes and subsequently extended to their pertinent methods and attributes. Following this, a complex vector entity is generated to represent the properties of each class property. The modulus of each complex number is then computed, taking into account the inherent traits of each given property, thus providing a comprehensive characterization of each class in the micro-architecture (Table 3).

Table 3 Assignment of unique angles to the design classes and classes' properties

Attribute / Method Name	Class	Angle (φ)	Angle (θ)	Angle (γ)
name	Runway	$k = 1 (1 \times \frac{\pi}{5})$	$q = 1 (1 \times \frac{\pi}{15})$	$\gamma = 1.1$
busy	Runway	$k = 1 (1 \times \frac{\pi}{5})$	$q = 2 (2 \times \frac{\pi}{15})$	$\gamma = 1.2$
is_busy()	Runway	$k = 1 (1 \times \frac{\pi}{5})$	$q = 3 (3 \times \frac{\pi}{15})$	$\gamma = 1.3$
set_busy()	Runway	$k = 1 (1 \times \frac{\pi}{5})$	$q = 4 (4 \times \frac{\pi}{15})$	$\gamma = 1.4$
get_name()	Runway	$k = 1 (1 \times \frac{\pi}{5})$	$q = 5 (5 \times \frac{\pi}{15})$	$\gamma = 1.5$
approach()	APlane	$k = 5 (5 \times \frac{\pi}{5})$	$q = 6 (6 \times \frac{\pi}{15})$	$\gamma = 5.6$
descend()	APlane	$k = 5 (5 \times \frac{\pi}{5})$	$q = 7 (7 \times \frac{\pi}{15})$	$\gamma = 5.7$
slow_down()	APlane	$k = 5 (5 \times \frac{\pi}{5})$	$q = 8 (8 \times \frac{\pi}{15})$	$\gamma = 5.8$
bypass()	APlane	$k = 5 (5 \times \frac{\pi}{5})$	$q = 9 (9 \times \frac{\pi}{15})$	$\gamma = 5.9$
receive()	AirMediator	$k = 4 (4 \times \frac{\pi}{5})$	$q = 10 (10 \times \frac{\pi}{15})$	$\gamma = 4.10$
ask_to_land()	AirMediator	$k = 4 (4 \times \frac{\pi}{5})$	$q = 11 (11 \times \frac{\pi}{15})$	$\gamma = 4.11$
done()	AirMediator	$k = 4 (4 \times \frac{\pi}{5})$	$q = 12 (12 \times \frac{\pi}{15})$	$\gamma = 4.12$
name	Tower	$k = 2 (2 \times \frac{\pi}{5})$	$q = 13 (13 \times \frac{\pi}{15})$	$\gamma = 2.13$
receive()	Tower	$k = 2 (2 \times \frac{\pi}{5})$	$q = 14 (14 \times \frac{\pi}{15})$	$\gamma = 2.14$
ask_to_land() (Inherited)	Tower	$k = 2 (2 \times \frac{\pi}{5})$	$q = 11 (11 \times \frac{\pi}{15})$	$\gamma = 2.11$
done() (Inherited)	Tower	$k = 2 (2 \times \frac{\pi}{5})$	$q = 12 (12 \times \frac{\pi}{15})$	$\gamma = 2.12$
name	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 15 (15 \times \frac{\pi}{15})$	$\gamma = 3.15$
mediator	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 16 (16 \times \frac{\pi}{15})$	$\gamma = 3.16$
approach() (Inherited)	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 6 (6 \times \frac{\pi}{15})$	$\gamma = 3.6$
descend() (Inherited)	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 7 (7 \times \frac{\pi}{15})$	$\gamma = 3.7$
slow_down() (Inherited)	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 8 (8 \times \frac{\pi}{15})$	$\gamma = 3.8$
bypass() (Inherited)	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 9 (9 \times \frac{\pi}{15})$	$\gamma = 3.9$
fly()	Plane	$k = 3 (3 \times \frac{\pi}{5})$	$q = 17 (17 \times \frac{\pi}{15})$	$\gamma = 3.17$

In Table 3, individual angles θ and φ are allocated to each class and its corresponding properties in the design. These angles are presented in their parametric forms. Following this, an angle γ is derived, which serves as the argument of the complex number representing the class property. Subsequently, Table 4 enumerates the attributes and additional methods utilized by each class method. The vector B_θ and its corresponding Hilbert mapping are then computed, as illustrated in the last column of Table 4. In Table 5, design properties employed within each class method are systematically identified and sorted. This arrangement enables the encoding of each property's presence or absence in the B_θ vector.

Subsequently, Table 6 provides a comprehensive summary of these design properties, categorizing them according to their respective class identities and presenting their corresponding final complex numbers in the last column. Notably, this final column serves as an accurate representation of the complex vector $\mathbf{d}$, encapsulating the totality of the design provided in Eq. 16.

Table 4 Used methods and attributes inside each class method of the design, calculation of B_θ vector, and inferring the complex number modulus

Class:Method	Uses Attributes	Uses Methods	B_θ	$\sqrt[n]{H_{(LN)}(B_\theta)} \cdot e^{i\varphi}$
Tower:receive	Plane: name	Plane: slow_down	[0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.72500934
Tower:ask_to_land	N/A	Runway:is_busy, Runway:set_busy, Plane:descend	[0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.92934417
Tower:done	Plane: name	Runway:set_busy	[0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.88109298
Plane:approach	Plane: name	Runway: get_name	[1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.75621196
Plane:descend	Plane: name	AirMediator: done	[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1]	1.75607784
Plane:slow_down	Plane: name	N/A	[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.75619855
Plane:bypass	Plane: name	AirMediator: ask_to_land	[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.75618515
Plane:fly	Plane: name	AirMediator: ask_to_land , Runway: get_name	[1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]	1.75617174

Table 5 A Sorted enumeration of utilized properties for encoding their presence in the B_θ vector, with each position matching a binary value in B_θ

Order	List of Used attributes and methods	Complex arguments (γ)
1	“Runway: get_name”	1.5
2	“Runway: is_busy”	1.3
3	“Runway: set_busy”	1.4
4	“Plane: approach”	3.6
5	“Plane: name”	3.15
6	“Plane: descend”	3.7
7	“Plane: slow_down”	3.8
8	“Plane: bypass”	3.9
9	“Plane: fly”	3.17
10	“Tower: name”	2.13
11	“Tower: receive”	2.14
12	“Tower: ask_to_land”	2.11
13	“AirMediator: receive”	4.10
14	“AirMediator: done”	4.12
15	“AirMediator: ask_to_land”	4.11

Table 6 Calculation of the final complex number that encodes each design property (attributes, methods, and their classes)

Attribute / Class Method	Class identity	Complex Number (z)
name	Runway	$1 \cdot e^{1.1i}$
busy	Runway	$1 \cdot e^{1.2i}$
name	Tower	$1 \cdot e^{2.13i}$
name	Plane	$1 \cdot e^{3.15i}$
mediator	Plane	$1 \cdot e^{3.16i}$
is_busy()	Runway	$1.07107548 \cdot e^{1.3i}$
set_busy()	Runway	$1.07107548 \cdot e^{1.4i}$
get_name()	Runway	$1.07107548 \cdot e^{1.5i}$
approach()	APlane	$1.07107548 \cdot e^{5.6i}$
descend()	APlane	$1.07107548 \cdot e^{5.7i}$
slow_down()	APlane	$1.07107548 \cdot e^{5.8i}$
bypass()	APlane	$1.07107548 \cdot e^{5.9i}$
receive()	AirMediator	$1.07107548 \cdot e^{4.10i}$
ask_to_land()	AirMediator	$1.07107548 \cdot e^{4.11i}$
done()	AirMediator	$1.07107548 \cdot e^{4.12i}$
receive()	Tower	$1.72500934 \cdot e^{2.14i}$
ask_to_land() (Inherited)	Tower	$1.92934417 \cdot e^{2.11i}$
done() (Inherited)	Tower	$1.88109298 \cdot e^{2.12i}$
approach() (Inherited)	Plane	$1.75621196 \cdot e^{3.6i}$
descend() (Inherited)	Plane	$1.75607784 \cdot e^{3.7i}$
slow_down() (Inherited)	Plane	$1.75619855 \cdot e^{3.8i}$
bypass() (Inherited)	Plane	$1.75618515 \cdot e^{3.9i}$
fly()	Plane	$1.75617174 \cdot e^{3.17i}$

Table 7 Subset of design properties along with corresponding complex numbers and B_θ vectors serving as filters on the enumerated list of used attributes and methods identified earlier in Table 5, the complex arguments of the used attributes and methods are also detailed in column 4

receive()	ask_to_land()	done()	complex arguments γ
$1.72500934 \cdot e^{2.14i}$	$1.72500934 \cdot e^{2.14i}$	$1.88109298 \cdot e^{2.12i}$	—
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1.5 \\ 1.3 \\ 1.4 \\ 3.6 \\ 3.15 \\ 3.7 \\ 3.8 \\ 3.9 \\ 3.17 \\ 2.13 \\ 2.14 \\ 2.11 \\ 4.10 \\ 4.12 \\ 4.11 \\ 0 \end{bmatrix}$

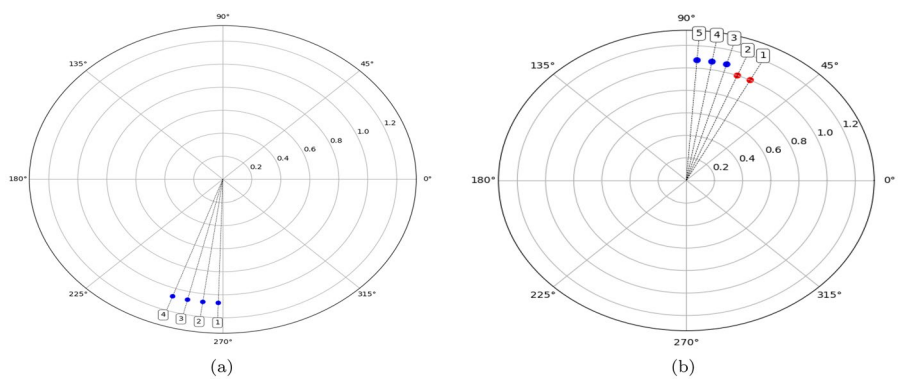


Fig. 8 2D illustration of complex numbers samples from the design: Red dots in (b) indicate attributes with a modulus of 1, while Blue dots signify other values. A modulus value of **1.07107548** signifies that the corresponding method's signature contains no arguments (special case where B_θ vector has all b_i are set to 0 s and only b_N (the last position of the vector) is set to 1)

$$d = \begin{bmatrix} 1 \cdot e^{1.1i} \\ 1 \cdot e^{1.2i} \\ 1 \cdot e^{2.13i} \\ 1 \cdot e^{3.15i} \\ 1 \cdot e^{3.16i} \\ 1.07107548 \cdot e^{1.3i} \\ 1.07107548 \cdot e^{1.4i} \\ 1.07107548 \cdot e^{1.5i} \\ 1.07107548 \cdot e^{5.6i} \\ 1.07107548 \cdot e^{5.7i} \\ 1.07107548 \cdot e^{5.8i} \\ 1.07107548 \cdot e^{5.9i} \\ 1.07107548 \cdot e^{4.10i} \\ 1.07107548 \cdot e^{4.11i} \\ 1.07107548 \cdot e^{4.12i} \\ 1.72500934 \cdot e^{2.14i} \\ 1.92934417 \cdot e^{2.11i} \\ 1.88109298 \cdot e^{2.12i} \\ 1.75621196 \cdot e^{3.6i} \\ 1.75607784 \cdot e^{3.7i} \\ 1.75619855 \cdot e^{3.8i} \\ 1.75618515 \cdot e^{3.9i} \\ 1.75617174 \cdot e^{3.17i} \end{bmatrix} \quad (16)$$

To elucidate the encoding mechanism underlying the B_θ vector, Table 7 features a selection of three class methods along with their respective modulus and arguments. Corresponding B_θ vectors, computed via the inverse Hilbert filling curve, are also presented. The final column showcases an ordered list derived from Table 5, which is further filtered through Eq. 13 utilizing B_θ . Finally, Fig. 8 offers a graphical representation of selected resultant complex numbers.

4.1 Metrics calculation

The metrics of the oriented object architecture can be directly calculated from the complex vector “d”, its calculation involves the reversal of the complex modulus and using it to filter and ordered subset λ . In the following sections, we calculate a number of oriented object metrics of our studied use case.

4.1.1 CBO metric

Coupling between objects (CBO) (Counsell et al. 2019; Chidamber and Kemerer 1994; Tempero and Ralph 2018), is a key metric in object-oriented programming that measures a class’s interdependence with other classes through method calls, attribute references, and inheritance. A high CBO value suggests increased coupling, making the code harder to maintain. CBO for a class ‘A’ is calculated as the sum of unique classes

directly linked to 'A,' excluding primitives and external libraries. As an illustration, consider the class "Tower" within the use case context, by encoding the argument with the angle $\gamma = 2 \cdot x$, where x denotes the identity of the Tower class properties, encompassing all attributes and methods, and subsequently inverting modulus of all the complex numbers that encode these properties, the methods "receive()", "ask_to_land()" and "done()" are observed to use distinct methods and attributes originating from two other classes (**Plane** & **Runway**). Consequently, $CBO(\text{Tower}) = 2$, in Table 7, the analysis of these three methods of the Tower class is given.

4.1.2 LCOM metric

In object-oriented design, the lack of cohesion in methods (LCOM) metric (Okike 2010; Ayyıldız and Haner Kırğılı 2021) serves as a quantitative measure of class cohesion. Class cohesion refers to the degree of relatedness between the methods and fields (or attributes) within a given class. A high degree of cohesion is generally considered advantageous, indicating that the class is well-structured and that each of its methods fulfills a distinct, well-defined role or responsibility. A lower numerical value of the LCOM metric is typically preferable, as it signals a more cohesive class structure. Several variations of the LCOM metric exist, but one widely recognized formulation of LCOM, introduced by Chidamber and Kemerer in their paper (Chidamber and Kemerer 1994) on object-oriented metrics, is defined as follows:

$$LCOM = \begin{cases} \frac{|\mathcal{P}| - |\mathcal{Q}|}{|\mathcal{P}|} & \text{if } |\mathcal{P}| > |\mathcal{Q}| \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

where:

- $|\mathcal{P}|$ is the number of method pairs that do not share instance variables (i.e., the number of disjoint method pairs),
- $|\mathcal{Q}|$ is the number of method pairs that share at least one instance variable.

The division by $|\mathcal{P}|$ in the formula is designed to normalize the difference between $|\mathcal{P}|$ and $|\mathcal{Q}|$.

In a demonstrative scenario, the analysis of Table 4 which includes the "Tower" and "Plane" classes methods and attributes, reveals that the lack of cohesion in methods (LCOM) metric indicates highly optimal results. The "Tower" class registers an LCOM value of **0.5**, attributed to the presence of two distinct method pairs lacking shared class fields $|\mathcal{P}| = 2$ and one method pair with at least one shared class instance $|\mathcal{Q}| = 1$. In contrast, the "Plane" class achieves a score of $LCOM = 0$, reflecting the total absence of method pairs without shared class fields.

5 Framework evaluation

5.1 Research questions

5.1.1 Generic framework

- **RQ1-1:** How effectively can the proposed framework integrate with multiple optimization algorithms compared to existing frameworks?
- **RQ1-2:** Does the integration of different optimization algorithms into the framework necessitate substantial adaptation?

5.1.2 Flexibility for refactorings

- **RQ2-1:** How comprehensive is the list of refactorings that the framework can handle?
- **RQ2-2:** Are there specific types or combinations of refactorings where the framework excels or struggles?

5.1.3 No solution representation

- **RQ3-1:** How does the proposed framework refactor without an explicit solution representation?
- **RQ3-2:** What impact does the absence of a solution representation have on the process of refactoring, and how does this compare with methods that rely on solution representations?

5.1.4 Metrics calculation

- **RQ4-1:** How comprehensive is the list of metrics that the framework can handle?
- **RQ4-2:** Are there specific metrics that are notably easier or more challenging to calculate within the framework?

Table 8 The initial particles are assigned to complex vectors, each subject to a slight random alteration achieved through the rotation of one of their constituent complex numbers (indicated by the bold, italic and bolditalic numbers)

P_1	P_2	...	P_N
$\begin{bmatrix} \mathbf{z_1} \\ \mathbf{z_2} \\ \dots \\ \mathbf{z_p} \end{bmatrix}$	$\begin{bmatrix} \mathbf{z_1} \\ \mathbf{z_2} \\ \dots \\ \mathbf{z_p} \end{bmatrix}$	...	$\begin{bmatrix} \mathbf{z_1} \\ \mathbf{z_2} \\ \dots \\ \mathbf{z_p} \end{bmatrix}$

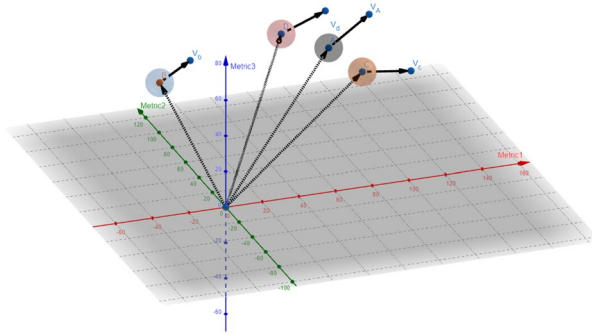


Fig. 9 Three-dimensional visualization portrays the arrangement of spheres within a three-dimensional space, which is constructed based on the metrics intended for optimization

5.1.5 Concurrency and scalability

- **RQ5-1:** Is the model conducive to optimization in concurrent settings?
- **RQ5-2:** How does the model's performance adapt to the growth of architecture and the intricacies of its elements?

5.2 Evaluation results

The presented model possesses the capacity to consolidate the representation of the architectural construct within a solitary vector, thereby permitting the manipulation of its internal structure to effectuate design refactoring. Upon the modification of this vector, both the overarching metric values and the architectural design undergo consequent alteration. This self-contained embodiment of the intricate vector serves to expedite its optimization through a spectrum of heuristic and optimization methodologies, obviating the necessity for concurrent adaptation of either the model or the optimization procedure itself. In the ensuing segments of this framework assessment, we expound upon the manner in which optimization of the design model can be achieved utilizing a trichotomy of distinct optimization methodologies.

5.2.1 Generic framework (RQ1-1 and RQ1-2)

- **Particle swarm optimization (PSO)**

To elucidate the proficiency of our artifact representation in seamlessly amalgamating with the PSO canonical algorithm, we postulate that our design is encapsulated within a complex vector d as illustrated by Eq. (18). For clarity and easier geometric visualization, we also suppose that we endeavor to optimize our design employing three distinct metrics: $M1$, $M2$, and $M3$, which can represent object-oriented

metrics. Previously, it has been ascertained that the geometric manifestation of our complex vector d materializes as a sphere centered at point O .

$$d = \begin{bmatrix} z_1 \\ z_2 \\ \dots \\ z_P \end{bmatrix} \quad (18)$$

We articulate the location of this center via coordinates x_{M_1} , y_{M_2} , and z_{M_3} in a 3D space, **each individual coordinate corresponds to the metric values associated with the complex vectors**, which have undergone random alterations. These altered complex vectors are attributed to each respective sphere (Particle), as elucidated in Fig. 9 and Table 8.

It is of paramount significance to expound that, despite the potential multifaceted intricacies inherent to the intricate vector, its spatial orientation within the designated exploratory domain is profoundly influenced by the nuanced variations of the quantifiable metrics. The methodical manipulation of these vectors, achieved through rotational operations applied to the complex entities, gives rise to alterations in metric attributes. These, in turn, give rise to modifications in the spatial disposition of each individual entity within the continuum of exploration. This procedural progression enables the entities to gradually traverse toward an optimal positional configuration that either maximizes or minimizes the objective function inherent to the quantifiable metrics.

$$\begin{aligned} \mathbf{v}_i(t+1) = & w \cdot \mathbf{v}_i(t) + c_1 \cdot \mathbf{r}_1 \cdot (\mathbf{p}_i(t) - \mathbf{x}_i(t)) \\ & + c_2 \cdot \mathbf{r}_2 \cdot (\mathbf{p}_{\text{gbest}}(t) - \mathbf{x}_i(t)) \end{aligned} \quad (19)$$

The starting velocities of our initial particles are randomly allocated. In subsequent iterations, these velocities are determined using Eq. 19. To guarantee precise velocity values, careful selection of the numerical scale for metric dimensions and algorithm parameters is imperative. For instance, by opting for 1/10 as a metric unit instead of 1, we enable Eq. 20 to yield the correct metric values.

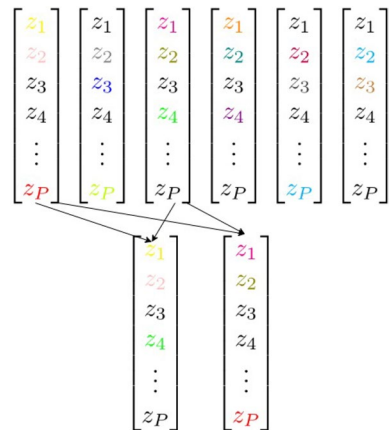
$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \quad (20)$$

For accurate position updates using Eq. 20, it's essential to consider that a position in a given dimension represents a metric. Thus, when updating the position, it's crucial to ensure that a corresponding complex rotation exists and aligns with the desired position (or metric). One of the benefits of employing the PSO algorithm in this scenario is that the cognitive parameters c_1 and c_2 as well as the weight w of the prior velocity, can be tailored based on metrics. **This allows us to prioritize one metric over others.** The comprehensive algorithm, in conjunction with our model, is provided in “Algorithm 1” that follows, where :

- $\mathbf{v}_{i,m}(t+1)$: Velocity of the i^{th} particle at time $t+1$ in dimension m (one of the metrics).
- $\mathbf{v}_{i,m}(t)$: Velocity of the i^{th} particle at time t in dimension m (one of the metrics).

- w_m : Inertia weight factor. It reflects the influence of the particle's prior velocity on its present velocity in dimension m .
- $\mathbf{x}_{i,m}(t)$: Position of the i^{th} particle at time t in dimension m .
- $\mathbf{x}_{i,m}(t + 1)$: Position of the i^{th} particle at time $t + 1$ in dimension m .
- $c_{1,m}$ and $c_{2,m}$: cognitive and social scaling coefficients dictate the relative impact of a particle's personal best and the swarm's global best positions on its new velocity, and these parameters may be modulated by vector metrics.
- $\mathbf{r}_1$ and $\mathbf{r}_2$: Random vectors generated for each dimension, often sampled uniformly between 0 and 1. They introduce randomness into the optimization process.
- $\mathbf{p}_{i,m}(t)$: Best known position of the i^{th} particle up to time t in dimension m .
- $\mathbf{p}_{\text{gbest}}(t)$: Best known position of all the particles in the swarm up to time t , known as the global best.

Fig. 10 A crossover after the third gene is performed on mutated genes (colored z_i)



Algorithm 1 Complex Vector Optimization using PSO Algorithm

```

1: Initialize swarm by rotating a randomly chosen complex number from each particle's constituting
   complex numbers.
2: for each particle in the swarm do
3:   Determine the particle's initial position by calculating its metrics.
4:   Set the particle's best-known position to its initial position.
5:   Determine the particle's velocity based on its initial metrics.
6: end for
7: Set the global best-known position using a random particle's position and its initial metrics.
8: while termination criteria is not met do
9:   for each particle in the swarm do
10:    for each metric  $m$  do
11:      Compute the new velocity for metric  $m$  as:

$$\mathbf{v}_{i,m}(t+1) = w_m \cdot \mathbf{v}_{i,m}(t) + c_{m,1} \cdot \mathbf{r}_1 \cdot (\mathbf{p}_{i,m}(t) - \mathbf{x}_{i,m}(t))$$


$$+ c_{m,2} \cdot \mathbf{r}_2 \cdot (\mathbf{p}_{\text{gbest},d}(t) - \mathbf{x}_{i,m}(t)) \quad (21)$$

12:      if there is a valid rotation leads to the new position/metric:  $\mathbf{x}_{i,m}(t+1)$  then
13:        Update the particle's position for metric  $m$ :

$$\mathbf{x}_{i,m}(t+1) = \mathbf{x}_{i,m}(t) + \mathbf{v}_{i,m}(t+1) \quad (22)$$

14:      end if
15:    end for
16:    if the new position of the particle has a superior fitness than its best-known position then
17:      Refresh the particle's best-known position with its new position.
18:    end if
19:    if the best position of the particle is fitter than the global best position then
20:      Update the global best position with the particle's best position.
21:    end if
22:  end for
23: end while
24: return the global best known position.

```

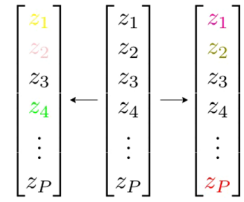
- **Genetic algorithm (GA)**

The proposed artifact representation also intrinsically supports integration with genetic algorithms, particularly when the population is modeled using genes and chromosomes. The contextualization of our optimization problem in this context will be as follows :

- **Gene** : a single complex number belongs to the vector
- **Chromosome** : the self contained complex vector
- **Fitness function** : object-oriented metrics of a complex vector

At the start of our optimization approach, we form an initial vectors population by randomly picking genes, represented as complex numbers in each vector, and rotating them based on the problem's constraints. Once this is done, parents are chosen from the population for crossover to produce the next generation of offspring. These

Fig. 11 The process of generating neighboring solutions by rotating one of the complex numbers of the vector



offspring are then subjected to mutations by randomly selecting and rotating their complex number (genes). In every iteration, the population is refreshed, and the best solutions are determined using a fitness function that evaluates optimal metrics.

Figure 10 offers a detailed exposition of the mutation procedure applied to the initial population to foster diversity, as well as the crossover mechanism enacted upon individual elements within the population. In 'Algorithm 2', we provide a comprehensive genetic algorithm, seamlessly amalgamated with our artifact's representational framework.

Algorithm 2 Genetic Algorithm Optimization of Complex Vectors

```

1: procedure GENETICALGORITHM(VectorsPopulationSize, maxGenerations)
2:   population  $\leftarrow$  initializePopulation(VectorsPopulationSize)
3:   generation  $\leftarrow$  0
4:   while generation < maxGenerations do
5:     newPopulation  $\leftarrow$  empty list
6:     for i  $\leftarrow$  1 to VectorsPopulationSize/2 do
7:       (parent1, parent2)  $\leftarrow$  selectParents(population)
8:       (child1, child2)  $\leftarrow$  crossover(parent1, parent2) //As described in figure 10.
9:       child1  $\leftarrow$  mutate(child1) //Randomly choosing a complex number and randomly rotate it.
10:      child2  $\leftarrow$  mutate(child2)
11:      add child1 and child2 to newPopulation
12:     end for
13:     population  $\leftarrow$  newPopulation
14:     generation  $\leftarrow$  generation + 1
15:   end while
16:   return bestVector(population)
17: end procedure
18: function INITIALIZEPOPULATION(VectorsPopulationSize)
19:   return list of size = populationSize of randomly initialized complex vectors
20: end function
21: function SELECTPARENTS(population)
22:   return two vectors selected based on fitness (metrics) from population.
23: end function
24: function CROSSEVER(parent1, parent2)
25:   Select crossover point(s) cp //As described in figure 10.
26:   Create child1 by combining parent1 before cp and parent2 after cp
27:   Create child2 by combining parent2 before cp and parent1 after cp
28:   return (child1, child2)
29: end function
30: function MUTATE(child)
31:   Randomly select a complex number from child and rotate its argument with respect to constraints.
32:   return mutated child
33: end function
34: function BESTVECTOR(population)
35:   return vector with highest fitness(metrics) from population.
36: end function

```

• Simulated annealing (SA)

In the pursuit of optimizing the complex vector through the employment of the Simulated Annealing optimization methodology, analogous to our previous utilization of the genetic algorithm, the genesis of our initial solutions is effectuated by subjecting a randomly chosen complex number from our artifact model to rotational manipulation. Subsequent to this initialization, the emulation of the annealing process is actualized by iteratively engendering neighboring solutions, achieved by enacting rotations upon individual complex numbers of the solutions vectors in each successive iteration as illustrated in Fig. 11. At the initial temperature $T(0)$, a larger subset of vector components is amenable to rotation. However, as the temperature evolves to an intermediate state $T(i)$, the rotation is confined to a select subset of components, modifiable via the parameter α as described in Eq. (23). This framework also permits the simultaneous initialization of multiple vectors to enter the SA optimization

Table 9 List of Refactorings in the Fowler's catalog (Fowler 1999)

Refactorings	Possibility	Refactorings by the framework
Pull up method	✓	Add a new complex number that encodes the new superclass and the method
Move method	✓	Rotate a complex number
Push down method	✓	Remove the complex number that encodes the method in the superclass
Pull up field	✓	Add a new complex number that encodes the new superclass and the field
Push down field	✓	Remove the complex number that encodes the field in the superclass
Extract class	✓	Rotate all the complex numbers that encode the old fields and methods
Move field	✓	Rotate a complex number
Inline class	✓	Rotate all the complex numbers that encode the old fields and methods
Collapse hierarchy	✓	Rotate all the complex numbers that encode the old fields of the subclass and the superclass
Extract superclass	✓	Add new complex numbers that encode the common features in a new superclass
Rename method	✗	—
Add parameter	✗	—
Extract interface	✓	Add new complex numbers that encode the class methods into a new interface
Encapsulate field	✗	—
Extract method	✗	—
Replace delegation with inheritance	✗	—
Replace inheritance with delegation	✗	—
Inline method	✗	—
Remove parameter	✗	—
Extract subclass	✓	Add new complex numbers that encode a set of features into a new subclass
Extract hierarchy	✗	—
Encapsulate collection	✗	—
Encapsulate downcast	✗	—
Hide method	✗	—

Table 9 (continued)

Refactorings	Possibility	Refactorings by the framework
Remove setting method	✗	–
Self encapsulate field	✗	–
Form template method	✗	–

process, thereby enhancing the likelihood of obtaining superior solutions. Algorithm 3 delineates a comprehensive SA algorithm that is intrinsically integrated into our artifact's representational framework.

Algorithm 3 Simulated Annealing for Multiple Complex Vectors

```

1: Initialization:
2: Define  $T_{\text{init}}, T_{\text{final}}, \alpha$  //The initial and final temperatures and rotations' control parameter.
3: Generate an initial set of complex number vectors
4: Compute  $E(v_i)$  for all vectors  $v_i$  //As described in section 5.2.1
5: Set  $T = T_{\text{init}}$ 
6: while  $T > T_{\text{final}}$  do
7:   for each vector  $v_i$  do
8:     Randomly select a complex number  $z_j$  from the vector
9:     Rotate  $z_j$  to get  $z'_j = z_j + \text{small angle}$ 
10:    Create new vector  $v'_i$  with  $z'_j$  in place of  $z_j$ 
11:    Compute  $E(v'_i)$ 
12:     $\Delta E = E(v'_i) - E(v_i)$ 
13:    if  $\Delta E < 0$  then
14:       $v_i = v'_i$ 
15:    else
16:      Accept  $v'_i$  with probability  $P(\text{accept}) = e^{-\Delta E/T}$ 
17:      if accepted then
18:         $v_i = v'_i$ 
19:      end if
20:    end if
21:  end for
22:   $T = \alpha \times T$ 
23: end while
24: Termination:
25: Return a subset of best vectors or optionally the best vector in the set

```

$$T(i) = T(0) \cdot \alpha^i \quad (23)$$

where:

- $T(i)$ is the temperature at iteration i .
- $T(0)$ is the initial temperature.
- α is the cooling schedule parameter.
- i is the current iteration.

$$P(\Delta E) = \exp\left(-\frac{\Delta E}{T}\right) \quad (24)$$

where:

- $P(\Delta E)$ is the probability of accepting a worse solution.
- ΔE is the difference in objective function values between the new solution and the current solution.
- T is the current temperature.

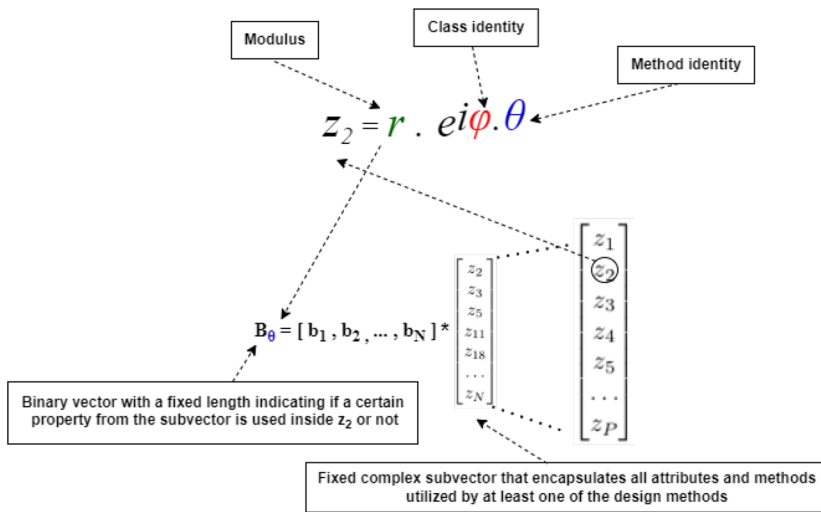


Fig. 12 The modulus of each complex number in the design vector encodes the B_θ vector, which, in turn, indexes the properties used by each property of the design. The term-by-term multiplication of B_θ with the fixed sub-vector of length N indicates which complex numbers are involved

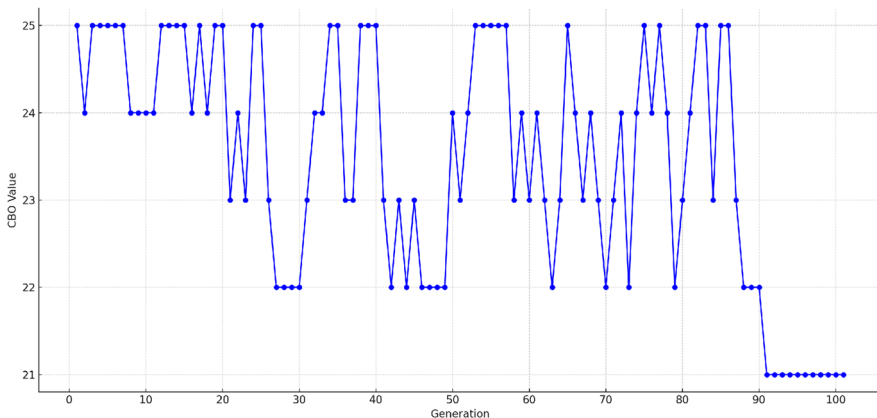


Fig. 13 The progression of the CBO metric during the genetic optimization process

The objective function E can be formulated through the integration of one or multiple object-oriented metrics that computation of which is supported by the proposed framework, while The differential ΔE serves to steer the optimization procedure through the calculation of the probability $P(\Delta E)$.

Table 10 A portion of the optimized vector resulting from rotational adjustments within the optimization procedure, with (d_i) denoting the initial vector and (d_f) signifying the final optimized vector

(d_i)	(d_f)
...	...
...	...
$1.64568183 \cdot e^{i \cdot 35.22}$	$1.64568183 \cdot e^{i \cdot 35.22}$
$1.68792224 \cdot e^{i \cdot 35.23}$	$1.68792224 \cdot e^{i \cdot 35.23}$
$1.61405274 \cdot e^{i \cdot 35.24}$	$1.61405274 \cdot e^{i \cdot 35.24}$
$1 \cdot e^{i \cdot 40.25}$	$1 \cdot e^{i \cdot 40.25}$
$1 \cdot e^{i \cdot 40.26}$	$1 \cdot e^{i \cdot 40.26}$
$1.13262554 \cdot e^{i \cdot 40.27}$	$1.13262554 \cdot e^{i \cdot 40.27}$
$1.19121701 \cdot e^{i \cdot 40.28}$	$1.19121701 \cdot e^{i \cdot 40.28}$
$1.11737441 \cdot e^{i \cdot 40.29}$	$1.11737441 \cdot e^{i \cdot 35.29}$
$1 \cdot e^{i \cdot 45.30}$	$1 \cdot e^{i \cdot 45.30}$
$1 \cdot e^{i \cdot 45.31}$	$1 \cdot e^{i \cdot 35.31}$
$1 \cdot e^{i \cdot 45.32}$	$1 \cdot e^{i \cdot 45.32}$
$1 \cdot e^{i \cdot 45.33}$	$1 \cdot e^{i \cdot 35.33}$
$1.31015497 \cdot e^{i \cdot 45.34}$	$1.31015497 \cdot e^{i \cdot 35.34}$
$1 \cdot e^{i \cdot 50.35}$	$1 \cdot e^{i \cdot 50.35}$
...	...
...	...

Table 11 Interpretation of the rotations proposed by the optimization process

Rotation	Interpretation
40.29° to 35.29°	Moving the method “checkout()” of the class “ShoppingCard” to the class “Order”.
45.31° to 35.31°	Moving the attribute “order” of the class “Payment” to the class “Order”.
45.33° to 35.33°	Moving the attribute “status” of the class “Payment” to the class “Order”.
45.34° to 35.34°	Moving the method “process_payment()” of the class “Payment” to the class “Order”.

5.2.2 Refactoring ability

RQ2-1, RQ2-2: Using the intermediate representation of the artifact, we can execute 44% of the refactorings from Fowler’s catalog (Fowler 1999). Table 9 delineates this in greater detail: the first and second columns provide a comprehensive list of Fowler’s refactorings that the framework can accommodate, while the third column elucidates how these refactorings are implemented within the framework. Nevertheless, due to the inherent characteristics of the framework, there are certain refactorings it cannot support. This constraint emerges due to the framework’s primary emphasis on providing extensive adaptability for modifying the foundational architectural elements of the design. It does not specifically cater to the quantitative attributes of the design components, such as line count, nomenclature, and data typing.

5.2.3 No solution representation is required

RQ3-1: In the proposed artifact representation, there is no requirement for a separate solution representation to facilitate refactoring. This is attributed to the dynamic nature of the framework. Specifically, it has been designed to allow intuitive manipulation through minor rotations of its constituent complex numbers. This stands in contrast to static artifact representations, such as abstract syntax trees and graphs (Qayum and Heckel 2009, 2011). In these static models, the artifact representation is initially analyzed, followed by the proposal of a sequence of refactorings (Ghannem et al. 2014) to modify it. Subsequently, the artifact representation undergoes evaluation. However, in our approach, the artifact representation concurrently serves as both the artifact and the solution.

To practically examine and demonstrate the ODR framework’s ability to directly refactor artifact representation without the necessity of a solution representation, we adopt a Python-oriented object design of an e-commerce management system comprising 15 classes, 39 attributes, and 27 methods. The computed CBO metric initially stood at 25. We structurally analyzed the code using sophisticated Python scripts to pull out methods, attributes, and interactions among the design’s properties. These were then used to create the initial complex vector, referred to as d_i (refer to Table 10). This vector is subsequently optimized using a genetic algorithm, the details of which are outlined below. For the sake of consistency in presentation, only portions of the complex vector are provided here. A comprehensive review of the generated complex vectors can be found in the project repository mentioned in the data availability section. To optimize our initial vector d_i we use the canonical genetic algorithm from the DEAP Python library (François-Michel 2023), the problem components mapping and the implementation details are the following:

Table 12 Collection of relevant and reproducible approaches utilizing separate solution representations, categorized by their artifact representation encoding type

Approach Category	Approach
AST-Based Approaches (ATBA)	Ghaith and Ó Cinnéide (2012), Hemati Moghadam and Ó Cinnéide (2011, 2012) Ó Cinnéide et al. (2011), O’Keeffe and Ó Cinnéide (2006, 2007)
Graph-Based Approaches (GBA)	Bowman et al. (2010), Di Penta et al. (2005), Di Pompeo and Tucci (2023), Ghannem et al. (2013), Griffith et al. (2011), Herold and Mair (2014), Kebir et al. (2017), Masoud and Jalili (2014), O’Keeffe and Ó Cinnéide (2004)

Table 13 Description of measured time durations for ODR Versus ATBA and GBA groups

Approach	Measured Time Duration
ODR	T1: Modify the unified vector T2: Compute the CBO Value
ATBA	T3: Modify the solution sequence T4: Apply the solution sequence T5: Compute the CBO Value
GBA	T6: Modify the solution sequence T7: Apply the solution sequence T8: Compute the CBO Value

- **Gene:** One complex number of the vector.
- **Chromosome:** The entire complex vector.
- **Population Size:** A group of 20 vectors. For creating a new individual within the population, a complex vector is selected randomly, and a rotation is applied while ensuring adherence to constraints.
- **Selection Method:** roulette wheel selection.
- **Crossover Operator and Rate:** Single-Point Crossover with a 0.6 probability, where the crossover location is randomly determined.
- **Mutation Operator and Rate:** A mutation involves one rotation of a complex number inside a vector, with a mutation rate of 0.2.
- **Replacement Strategy:** Elitist Strategy.
- **Termination Criteria:** Reaching a limit of 500 generations.

Moreover, we designed the fitness function ' Δ ' which directly computes the CBO metric from the complex vector ' d '. The definition of this function is described by the following equation:

$$\Delta(d) = \begin{cases} 0 & \text{if } \arg(z_i) \times \arg(z_j) < 0 \\ \sum_{i=1}^P \sum_{j=1}^N u(\arg(z_i) - \arg(z_j))^2 & \text{otherwise.} \end{cases} \quad (25)$$

where N and P are precisely the constants defined in Sects. 3.1.1 and 3.2, N is the length of the vector B_θ that encodes the properties used by a certain method in the modulus of its corresponding complex number, and P is the total number of properties (methods and attributes) in the design. Finally, ' u ' is the function defined by:

$$u(x) = \begin{cases} 0 & \text{if } |x| < 1, \\ 1 & \text{otherwise.} \end{cases} \quad (26)$$

The quantity $\arg(z_i)$ represents the arguments of complex numbers that constitute the vector d_i intended for optimization, while $\arg(z_j)$ denotes the arguments of the complex numbers indexed by the modulus of each complex number in the vector

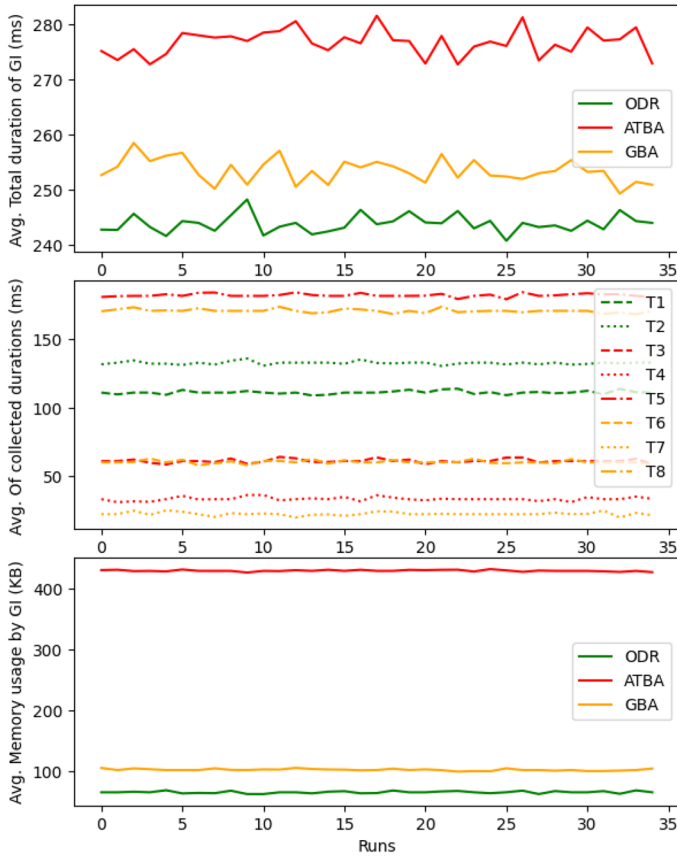


Fig. 14 Average metrics of the collected execution data for each approach over 35 runs

d_i . As elaborated in Sect. 3.1.1, each method's modulus indexes a set of complex numbers. These complex numbers, in turn, encode attributes and methods used by the property itself. This mechanism is further clarified in Fig. 12. The function Δ is designed in such a way that if the arguments of the used properties $\arg(z_j)$ (encoded by the modulus) match the argument of the currently analyzed property $\arg(z_i)$, the CBO value is 0. Otherwise, it gives 1, indicating that the current property is using properties from other classes, resulting in tight coupling.

Experimentally, we conducted the genetic optimization process, permitting the algorithm to exclusively move properties from one class to another through the rotation of the complex numbers in the vector. The genetic process converges to a minimal value of CBO around the 90th generation, as depicted in Fig. 13.

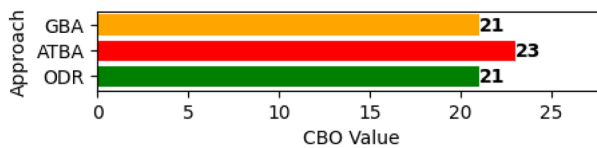
The resulting optimized vector (refer to Table 10) suggests rotating a group of 4 complex numbers. Through these rotations, the overall CBO value of the design

Table 14 Comparison of Qualitative Attributes Among ODR, ATBA, and GBA Approaches

Attributes	ODR	ATBA	GBA
Solution Representation	Not required	Often Required	Often Required
Genericity ¹	High	Limited	Limited
Memory Usage	Low	High	Moderate
Objects Associations Encoding	✓	✓	✓
Encapsulation States Encoding	✗	✗	✗
Inheritance Tracking	✓	✓	Dependent on Context
Interface Implementing Tracking	✓	✓	Dependent on Context
Method Decorators Tracking	✗	✓	✗
Method Signature Analysis	✓	✓	✓
Code Semantics and Syntax Analysis	✗	Dependent on Context	✗
Explainability of Refactorings	Limited	Limited	Moderate
Refactoring Ability	High	Limited	Moderate
Code Behavior Preservation ²	Limited	Dependent on context	Limited

¹Minimal need for verbose encoding to dictate how refactorings should be executed within the heuristic

²Automatically ensures that any transformations proposed by the refactoring process maintain the original behavior and of the software (Mariani and Vergilio 2017)

**Fig. 15** Optimal minimum CBO values achieved by each refactoring approach

is minimized to a value of 21. These rotations are decoded in Table 11, providing insight into the optimal refactorings for minimizing the CBO value of the design.

RQ3-2: The absence of a solution representation yields a significant advantage: the seamless integration of optimization heuristics. Given the model's capacity for intuitive and swift refactoring, there is minimal need for verbose encoding to dictate how refactorings should be executed within the heuristic. This is because rotating a complex number involves a straightforward mathematical addition, which the heuristic can easily perform. Furthermore, the heuristic is not burdened with generating a solution distinct from the input (the artifact representation). This is beneficial as there is no necessity to modify the heuristic to produce a specific solution type. Instead, it merely modifies the artifact representation directly, evaluates it, and subsequently returns it in its original format. Another notable advantage is the ease of rolling back modified artifacts. When long 'refactoring sequences' are applied to the artifact, reverting certain modifications, especially during intermediate stages of optimization where new solutions are explored, becomes challenging. Hence, minor modifications, such as rotations, can be conveniently stored in ordered arrays. These

arrays can subsequently be analyzed, offering potential insights during post-optimization evaluations.

Furthermore, as previously discussed in this paper, a significant limitation of SBR methodologies that utilize an independent solution representation is the necessity for implementing the proposed suboptimal solutions on the artifact representation at each optimization step. This implementation is essential to evaluate these solutions and calculate their fitness values. The calculation of the fitness value, however, occurs externally to the optimization process. This external evaluation introduces substantial delays in the optimization cycle and increases its memory requirements, particularly when managing complex and large-scale code designs. In our methodology, we tackled this limitation by integrating the artifact directly into the solution representation, thereby enabling the optimization process to function as a single cohesive entity without reliance on external validation. This integration, while resulting in a more extended solution representation, significantly enhances the efficiency of the optimization cycle in terms of execution time and memory usage.

To empirically examine these enhancements, we compiled various approaches that utilize a separate solution representation, such as sequences, sets, or vectors of refactorings. Typically, these methodologies represent the artifact as an AST or a graph. The objective is to identify the optimal sequence of refactorings that improves the artifact. We have categorized these approaches in Table 12, presented below.

Subsequently, we reproduced the optimization process component of these approaches, in which the heuristic iteratively modifies the solution representation and applies it to the artifact representation to compute the fitness value. To accomplish this, we encoded the object associations, attributes, and methods of our examined e-commerce design into an AST utilizing the built-in Python library “ast”, and into a graph via the “networkx” Python library. These artifact representations are also available in the project repository. We adopted a solution representation in the form of a sequence of six elements, $S = \{(c_i, a_i, c'_i)\}_{i=1}^6$, where each element is a tuple consisting of three components. To ensure consistent comparison, the optimization process was restricted to allow the heuristic to only move methods or attributes from one class to another. Thus, the first element of the tuple, c_i , is the identifier of the class currently holding the method/attribute, a_i is the identifier of the method/attribute, and c'_i is the destination class.

Adhering to the established best practices for conducting statistical tests on randomized algorithms, as detailed in Arcuri and Briand (2011), we conducted the replicated optimization processes for both the ATBA and GBA groups of approaches, along with the previously described ODR optimization process, a total of 35 times using the canonical genetic algorithm, consistent with the implementation details outlined in the preceding section. During these executions, for each genetic iteration (GI) –defined as the production of a new generation– we collected data on memory usage and eight durations $\{(T_1, \dots, T_8)\}$ from each approach’s execution process, which are detailed and categorized in Table 13. Regarding the objective function, we reused the fitness function outlined in Eq. 25 for the ODR. Additionally, we relied on the formal definition of the CBO

metric provided in Tempero and Ralph (2018) to implement the fitness function that calculates the CBO metric directly for the AST and the graph representing the artifact to optimize.

The collated and analyzed execution data is depicted in Fig. 14 above. The first graph delineates the average total duration of a single GI for each run, offering a macroscopic view of the temporal efficiency per iteration. The second graph provides a more granular analysis by deconstructing the total duration of a single GI into two sub-durations for the ODR framework and three sub-durations for the comparative approaches. The third graph renders the average memory usage per GI for each run, thereby illustrating the memory demand associated with each approach throughout the optimization process.

Although all three approaches successfully optimized the artifact's CBO metric to a proximate value around 22, as delineated in Fig. 15, the ODR approach is noted for its superior efficiency in terms of time and memory utilization. Despite the durations T1 and T2 for ODR being higher than those of the other approaches—with the exception of T8 and T5, which correspond to the computation of the fitness value—this is attributable to the more extensive vector used by ODR. However, ODR counterbalances this by eliminating the requirement for applying modifications to the artifact representation, given that, within the ODR methodology, the artifact representation inherently serves as the solution representation. Cumulatively, the total duration of the GIs in ODR is significantly reduced compared to those of the other approaches. In terms of memory utilization, the ODR approach utilizes a condensed vector to encapsulate artifact attributes, which not only enables the direct computation of fitness values but also simultaneously serves as the solution representation. In contrast, the ATBA and GBA approaches adopt memory-intensive strategies, utilizing sophisticated memory structures to compute the fitness values for the solution sequences within each iteration, this strategy significantly increases their cumulative memory requirements. To provide a comprehensive overview, we have included a summarized comparison of the key qualitative attributes between ODR and other groups of approaches in Table 14.

5.2.4 Metrics

RQ4-1, RQ4-2: Metrics play a pivotal role in the design optimization process, as they underpin the construction of the fitness function. This function may utilize a single metric or a combination thereof. As explained in the previous sections, the framework model proficiently manages metrics that are intrinsically linked to the architecture and design of the artifact, such as cohesion metrics [e.g., LCOM, cohesion among methods of classes (CAMC) (Chahal and Singh 2010)], coupling metrics [e.g., afferent couplings (CA), efferent couplings (CE) (Santos et al. (2017), CBO], and certain size metrics like number of children (NOC) (Sandhu and Singh 2005) and depth of inheritance tree (DIT) (Gulia and Chillar 2021). However, the artifact representation encounters challenges with metrics that correlate directly with the number of code lines, encapsulation states, and behavioral facets of the design, such as data access metric (DAM) (Xenos et al. 2000) and response for class (RFC) (Cheikhi et al. 2014).

5.2.5 Concurrency and scalability

RQ5-1: In parallel environments, optimization can be executed independently for each complex vector. Given that each vector is **self-sufficient**, encompassing all necessary data for optimization and metric assessment, this approach can be particularly beneficial in scenarios with extensive architectural sizes. Such a configuration has the potential to reduce computational burden and produce competitive outcomes. **RQ5-2:** Within a design framework comprising multiple classes and properties, the framework model demonstrates commendable scalability. In our approach, scalability predominantly involves appending new complex numbers to the vector. Adjusting the arguments of these complex numbers is relatively straightforward, primarily entailing increments to existing angles. However, the calculation of the modulus warrants attention. This is because the modulus encodes the arguments and methods utilized by the method corresponding to that particular complex number. Nevertheless, the vector B_θ represents effectively used arguments and methods within a method as '1 s', while denoting unused arguments and methods (though they might be employed by other methods) as '0 s'. Given that a significant portion of the B_θ elements will be '0 s', this characteristic can be leveraged for compression prior to computing its Hilbert distance. Such an approach not only ensures that the model scales proportionally with the design but also maintains its compactness and self-sufficiency.

6 Threats to validity

Internal validity: While our framework effectively integrates with three distinct optimization heuristics, the discrete character of artifact manipulation via set quantities of rotations might complicate its alignment with heuristics requiring continuous search space exploration. Additionally, it is imperative to highlight that the model does not encode the encapsulation level of attributes. Consequently, any refactoring efforts targeting attributes with encapsulation levels other than 'public' could potentially render the results invalid from both programming and design architectural perspectives.

Limitations: In situations where a method necessitates a significant number of arguments, there may be a need to optimize the computation of the vector B_θ , this is due to the potential magnitude of the decimal component of the modulus of the complex number which could reach a threshold where inconsistencies arise. It is worth noting that the model was originally designed to encode class methods with signatures accommodating no more than **20 arguments**. Furthermore, the model's inability to compute certain metrics, which have a direct correlation with the number of lines of code, encapsulation states, and specific behavioral aspects of the design, such as LCO, DAM, and RFC, poses a limitation. Consequently, the optimal solution derived from the optimization process may not be wholly representative of these particular metrics. To address this, it might be prudent to concurrently optimize these metrics using an auxiliary process, ensuring a more comprehensive representation in the optimized model. Additionally, it is essential to note that the

model does not encode specific properties, including class templates, method templates, and method decorators.

7 Related work

In the realm of SBR, only a limited number of approaches aim to adopt a unified representation for directly optimizing the artifact (Mariani and Vergilio 2017). This is mainly due to the challenges in translating designs into optimizable formats. These strategies often utilize ASTs, graphs, and tree-like structures, which require significant computational resources. To address these challenges, such methods typically encode only specific elements of the design, simplifying and reducing the depth and dependencies of these structures.

7.1 Unified artifact and solution representations in SBR

Tawosi et al. (2015) developed an innovative approach for the automated generation and optimization of software designs, employing ant colony optimization (ACO). This methodology involves extracting design elements from requirements and use-case documents through the use of NLP techniques and Semantic Networks. Following this, an artifact representation is generated in the form of a directed acyclic graph (DAG), which is subsequently optimized using the heuristic. Contrary to our approach, the approach by Tawosi et al. does not integrate some object-oriented design concepts such as inheritance and abstraction, nor does it discuss the potential for refactoring already elaborated software designs. Notably, their methodology is specifically designed around the ACO heuristic. In their studies, O’Keeffe and Ó Cinnéide (2004, 2003) tackle the SBR problem using combinatorial optimization techniques. They utilize a tool named Dearthoir to directly enhance the object-oriented design by evaluating metrics related to method cohesion, class coupling, and how the inheritance hierarchy is structured. Yet, the specifics of how the tool represents designs are not detailed in the study. Additionally, this method overlooks the encoding of certain design aspects, such as the arguments and dependencies of methods.

Mahouachi et al. (2012), adopt a significantly different approach to identifying and correcting software design defects using genetic programming (GP) and lists of refactorings. In this context, the GP framework encodes IF-THEN rules within trees, where the IF part represents conditions based on software metrics for detecting design defects, and the THEN part suggests specific refactorings aimed at correcting these defects. The optimization process within the GP framework aims to evolve these trees to improve their effectiveness in defect detection and correction. Notably, this method doesn’t directly encode the design artifacts. Instead, it encodes modifications applied to a set of example designs and uses this data to guide the detection and correction of defects. This reliance on specific examples limits how extensively it can explore potential solutions. Similarly, Wongpiang and Muenchaisri (2013) focuses on representing the dynamic states of the software design as it undergoes

various refactoring techniques, rather than detailing the design itself, while the study does not explicitly discuss the mechanism of applying these refactorings, the core of their methodology lies in optimizing the refactoring process using the greedy algorithm. This algorithm selects the most beneficial sequence of refactorings by evaluating the potential maintainability improvements that each sequence's path offers. Furthermore, Hemati Moghadam et al. (2015) and Liu et al. (2008) focus on the automated scheduling of refactorings to improve software design while addressing conflicts and dependencies among refactoring operations. They use UML diagrams and graphs for a more abstract design representation. The objective is to refine the refactoring sequence by acknowledging these conflicts and dependencies, aiming for an improved final design. However, the choice of specific refactorings to apply is not automated by the optimization process, they are predefined or selected by developers.

7.2 Approaches separating solution and artifact representations

In contrast, the majority of studies adopt separate representations for artifacts and solutions, mainly because optimizing a straightforward and linear structure like a vector or a sequence of actions is easier than optimizing the whole design representation.

7.2.1 Sequence of refactorings as a solution representation

In a recent study (Ahmadi et al. 2022), the artifact is represented as a 26-element vector capturing software metrics, while the solution representation is a sequence of refactoring actions determined by an agent trained using deep Q-learning. Similarly, Abid et al. (2021) uses the software codebase as the artifact and a sequence of refactorings as a solution representation, emphasizing refactoring dependencies and the application of intelligent change operators. A sequence of refactorings is also used to encode the solution representation in Ghannem et al. (2013), where the artifact representation is the UML class diagram. The sequence is used to align the initial and revised diagrams. These refactorings, influenced by designer feedback, are constrained to ensure coherence and prevent conflicts in the design model. Comparably, Mkaouer et al. (2014) employs UML design models as the artifact, with the solution being a sequence of refactoring operations applied to identified actors within the UML design. Furthermore, in their study (Ghannem et al. 2014), the authors tackle the challenge of model design quality in model-driven engineering through refactoring. The artifact representation is the initial model that requires refactoring, characterized by its design and structural metrics. The solution representation is also conceptualized as a sequence of refactoring operations. These operations are derived to maximize the structural similarity between the initial model and a set of example models. By treating refactoring as a combinatorial optimization problem, the study aims to generate effective refactoring suggestions, leveraging a genetic algorithm to explore and identify optimal refactoring sequences. Analogy, Kebir et al. (2017) addresses the refactoring of component-based software systems that accumulate

bad smells over time. The artifact representation is the source code model of the software, highlighting its design, structure, and associated bad smells. The solution representation is an optimal sequence of refactorings aimed at reducing these bad smells. To determine this sequence, a genetic algorithm is employed, guided by a catalog of component-relevant refactorings and bad smell detection rules, aiming to enhance the software's design and maintainability. Another approach (Ouni et al. 2013) in the same direction where the authors address the challenge of refining software design through refactoring, specifically targeting bad smells in the code. The artifact representation is the software's design, characterized by its accumulated design anomalies. The solution representation is an optimal sequence of refactorings, determined using the non-dominated sorting genetic algorithm (NSGA-II).

7.2.2 Other forms of solution representations

Graph transformation rules are also used as solution representations, in Qayum and Heckel (2009), the authors employ graphs to represent software architectures at the class level, serving as the artifact representation. This graph representation captures the intricate structure and relationships within the software. As for the solution representation, the study utilizes graph transformations to formally outline refactoring operations on these software architectures. By harnessing this approach, the authors can systematically identify and apply refactoring steps, optimizing the software's structure and functionality. Similarly, the authors in Qayum and Heckel (2011), present a novel approach to software refactoring. The artifact representation is object-oriented designs, captured as graphs encoded in the GXL format. This representation emphasizes the design's structure and dependencies. The solution representation revolves around graph transformation rules that detail refactoring operations. By employing the unfolding of a graph transformation system and the ant colony optimization metaheuristic, the study seeks to optimize refactoring sequences, enhancing software design through a combination of graph transformation theory and heuristic search. In another approach, Alkhazi et al. (2016) uses the ATLAS transformation language (ATL) for artifact representation, focusing on model-to-model transformation, the solutions are represented by set of rules that can be merged and optimized in a sequential manner, and Mokaddem et al. (2018) presents metamodels as the artifact representation, encoded using the Jess language. The solution representation in this work revolves around Jess-encoded refactoring rules, which guide the conditions for applying specific refactoring operations on metamodels.

7.3 Conclusion of related work

A consistent theme observed across the majority of referenced studies is their employment of distinct artifact and solution representations when tackling the refactoring conundrum. These studies necessitate an explicit encoding of a predetermined

set of refactorings. Subsequently, a solution representation is furnished to the optimization heuristics, facilitating the application and evaluation of refactorings to ascertain the optimal set. While the encoding methodologies for these artifact and solution representations exhibit variations across different studies, the foundational principle remains invariant. Contrarily, our approach coalesces these representations into a singular, intuitive, and dynamic format. This integrated representation is directly leveraged by the heuristics to enhance the design, thereby streamlining design manipulation and expediting solution generation at the same time.

8 Conclusion and future work

In conclusion, this research introduced and examined an innovative artifact representation, facilitating the intuitive manipulation, optimization, and evaluation of object-oriented design without necessitating a solution representation. The model's genericity, and the study's hypothesis, were validated against three distinct search heuristics: the canonical genetic algorithm, simulated annealing, and particle swarm optimization. The practical application of the intuitive design manipulation and the straightforward calculation of design metrics directly from the artifact representation were demonstrated through a real-world design refactoring scenario. Notably, the model enables the application of approximately 44% of Fowler refactorings and the computation of a significant array of object-oriented metrics. These capabilities assist search heuristics in optimizing the design with precision, granting them substantial latitude in design manipulation. Furthermore, the inherent self-sufficiency of the artifact representation permits optimization in parallel environments, potentially alleviating computational demands and yielding competitive results.

In prospective research endeavors, attention could be directed towards augmenting the model with advanced features, such as encoding encapsulation levels within the complex numbers representing the attributes. There is also potential to expand the model to accommodate the computation of the residual set of object-oriented metrics and to facilitate additional refactoring techniques. Moreover, the incorporation and assessment of diverse heuristic types warrant further exploration. The modulus of the complex numbers, which symbolize the attributes and methods used by a certain class method, may also be revisited with the possibility of applying innovative dimension reduction and compression methodologies to its computation.

Author Contributions The co-authors, Tarik Houichime and Younes El Amrani, made equivalent intellectual contributions to this study. They were jointly engaged in the study's genesis, devising its methodology, and formulating the framework for optimization of design refactoring. Furthermore, the authors collectively undertook the examination and explication of the findings, as well as the composition and refinement of the manuscript.

Funding This study was conducted without any external financial support or funding.

Data availability The repository, accessible at [ODR research repository](#), encompasses the design architecture of an e-commerce management system along with the complex vectors, graph, and AST that were analyzed and optimized in the course of this study.

Declarations

Conflict of interest The authors, Tarik Houichime and Younes El Amrani, unequivocally declare that no Conflict of interest, either financial or nonfinancial, have influenced the research and development of this work. The authors affirm that they have conducted this study with the utmost objectivity and academic integrity, free from any Conflict of interest that could potentially compromise the impartiality and validity of this research findings.

Experimental setup All experiments were conducted on a standardized test environment to ensure consistency in performance evaluation. The computational platform utilized was a laptop equipped with an Intel Core i5 processor and 8 GB of RAM from the 8th generation. This hardware configuration was selected to demonstrate the practical applicability and efficiency of the algorithms under typical use conditions.

References

- Abid, C., Ivers, J., Ferreira, T.D.N., Kessentini, M., Kahla, F.E., Ozkaya, I.: Intelligent change operators for multi-objective refactoring. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE); (2021). pp. 768–780
- Agnihotri, M., Chug, A.: Understanding refactoring tactics and their effect on software quality. In: 2022 12th International Conference on Cloud Computing, Data Science & Engineering (Confluence); pp. 41–46. (2022)
- Agnihotri, M., Chug, A.: A systematic literature survey of software metrics, code smells and refactoring techniques. *J. Inf. Process. Syst.* **16**(4), 915–934 (2020). <https://doi.org/10.3745/JIPS.04.0184>
- Ahmadi, H., Ashtiani, M., Azgomi, M.A., Saheb-Nassagh, R.: A DQN-based agent for automatic software refactoring. *Inf. Softw. Technol.* **147**, 106893 (2022). <https://doi.org/10.1016/j.infsof.2022.106893>
- Alkhazi, B., Ruas, T., Kessentini, M., Wimmer, M., Grosky, W.: Automated refactoring of ATL model transformations: a search-based approach. In: The ACM/IEEE 19th International Conference. (2016); p. 304. <https://doi.org/10.1145/2976767.2976782>
- Amal, B., Kessentini, M., Bechikh, S., Dea, J., Said, L.B.: On the use of machine learning and search-based software engineering for ill-defined fitness function: a case study on software refactoring. In: Le Goues C., Yoo S., (Eds.) *Search-Based Software Engineering*. Lecture Notes in Computer Science. Cham: Springer International Publishing. pp. 31–45. (2014)
- Anders, S.: Visualization of genomic data with the Hilbert curve. *Bioinformatics* **25**(10), 1231–1235 (2009). <https://doi.org/10.1093/bioinformatics/btp152>
- Arcuri, A., Briand, L.: A practical guide for using statistical tests to assess randomized algorithms in software engineering. In: Proceedings of the 33rd International Conference on Software Engineering. ICSE '11. New York, NY, USA: Association for Computing Machinery; (2011). pp. 1–10
- Ayyıldız, T., Haner Kırıl, E.: Analysis of lack of cohesion in methods (LCOM): a case study. In: 2021 2nd International Informatics and Software Engineering Conference (IISEC). (2021), <https://doi.org/10.1109/IISEC54230.2021.9672419>
- Bowman, M., Briand, L.C., Labiche, Y.: Solving the class responsibility assignment problem in object-oriented analysis with multi-objective genetic algorithms. *IEEE Trans. Software Eng.* **36**(6), 817–837 (2010). <https://doi.org/10.1109/TSE.2010.70>
- Bringmann K., Cassis A., Fischer N., Künnemann M. Fine-grained completeness for optimization. In: P. International Workshop and International Workshop on Approximation, Randomization, and Combinatorial Optimization Algorithms and Techniques. (2021 Jul), <https://doi.org/10.48550/arXiv.2107.01721>. arxiv:2107.01721. [cs]

- Chahal, K., Singh, H.: Exploring design level class cohesion metrics. *JSEA*. **3**, 384–390 (2010). <https://doi.org/10.4236/jsea.2010.34043>
- Cheikhi, L., Al-Qutaiha, R., Idri, A., Sellami, A.: Chidamber and Kemerer object-oriented measures: analysis of their design from the metrology perspective. *Int. J. Softw. Eng. Appl.* **8**, 359–374 (2014)
- Chidamber, S.R., Kemerer, C.F.: A metrics suite for object oriented design. *IEEE Trans. Software Eng.* **20**(6), 476–493 (1994). <https://doi.org/10.1109/32.295895>
- Costa, A., Cappadonna, F.A., Fichera, S.: A dual encoding-based meta-heuristic algorithm for solving a constrained hybrid flow shop scheduling problem. *Comp. Ind. Eng.* **64**(4), 937–958 (2013). <https://doi.org/10.1016/j.cie.2013.01.004>
- Counsell, S., Arzoky, M., Destefanis, G., Taibi, D.: On the relationship between coupling and refactoring: an empirical viewpoint. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), Porto de Galinhas, Recife, Brazil: IEEE, pp. 1–6. (2019)
- Cromwell, H.C., Mears, R.P., Wan, L., Boutros, N.N.: Sensory gating: a translational effort from basic to clinical science. *Clinical EEG Neurosci.* **39**(2), 69–72 (2008)
- Di Penta, M., Neteler, M., Antoniol, G., Merlo, E.: A language-independent software renovation framework. *J. Syst. Softw.* **77**(3), 225–240 (2005). <https://doi.org/10.1016/j.jss.2004.03.033>
- Di Pompeo, D., Tucci, M.: Multi-objective software architecture refactoring driven by quality attributes. *arXiv.org*. (2023), <https://doi.org/10.48550/arXiv.2301.07500>. arxiv:2301.07500. [cs]
- Douglas, R.H., Gödel, E.: *Bach an Eternal Golden Braid by Douglas R. Hofstadter*. Addison-Wesley Professional, Boston (1999)
- Fowler, M.: *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Longman Publishing Co., Inc., USA (1999)
- François-Michel, D.R.: *Evolutionary computation framework for rapid prototyping and testing of ideas*. github. (2023)
- Gad, A.G.: Particle swarm optimization algorithm and its applications: a systematic review. *Archiv. Comput. Methods Eng.* **29**(5), 2531–2561 (2022). <https://doi.org/10.1007/s11831-021-09694-4>
- Ghaith, S., Ó Cinnéide, M.: Improving software security using search-based refactoring. *Int. Symp. Search Based Softw. Eng.* **7515**, 135 (2012). https://doi.org/10.1007/978-3-642-33119-0_10
- Ghannem, A., El Boussaidi, G., Kessentini, M.: Model refactoring using interactive genetic algorithm. In: Ruhe G., Zhang Y., (Eds.). *Search Based Software Engineering. Lecture Notes in Computer Science*. Berlin, Heidelberg: Springer. pp. 96–110. (2013)
- Ghannem, A., El Boussaidi, G., Kessentini, M.: Model refactoring using examples: a search-based approach. *J. Softw. Evol. Process.* **26**(7), 692–713 (2014). <https://doi.org/10.1002/smr.1644>
- Griffith, I., Wahl, S., Izurieta, C.: Evolution of legacy system comprehensibility through automated refactoring. In: *Proceedings of the International Workshop on Machine Learning Technologies in Software Engineering*. (2011)
- Gulia, P., Chillar, R.: New proposed inheritance metrics to measure the software complexity. *Int. J. Comput. Appl.* (2012). <https://doi.org/10.5120/9403-3486>
- Gungor, M.: De-Moivre and Euler formulae for dual-complex numbers. *Univ. J. Mathe. Appl.* **2**, 126–129 (2019). <https://doi.org/10.32323/ujma.587816>
- Hemati Moghadam, I., Ó Cinnéide, M.: Automated refactoring using design differencing. In: *Proceedings of the Euromicro Conference on Software Maintenance and Reengineering, CSMR*. p. 52. (2012) <https://doi.org/10.1109/CSMR.2012.15>
- Hemati Moghadam, I., Ó Cinnéide, M.: Code-Imp: A tool for automated search-based refactoring. In: *Proceedings - International Conference on Software Engineering*. (2011) <https://doi.org/10.1145/1984732.1984742>
- Hemati Moghadam, I., Ó Cinnéide, M.: Resolving conflict and dependency in refactoring to a desired design. *e-Infom. Softw. Eng. J.* **9**, 37–56 (2015). <https://doi.org/10.5277/e-Inf150103>
- Herold, S., Mair, M.: Recommending refactorings to re-establish architectural consistency. In: Avgeriou P., Zdun U., (Eds.). *Software Architecture. Lecture Notes in Computer Science*. Cham: Springer International Publishing; (2014). pp. 390–397
- Immanuel, S.D., Chakraborty, U.K.: Genetic algorithm: an approach on optimization. In: *2019 International Conference on Communication and Electronics Systems (ICCES)*. pp. 701–708. (2019)
- Imran, M., Hashim, R., Khalid, N.E.A.: An overview of particle swarm optimization variants. *Proc. Eng.* **53**, 491–496 (2013). <https://doi.org/10.1016/j.proeng.2013.02.063>
- Katoch, S., Chauhan, S.S., Kumar, V.: A review on genetic algorithm: past, present, and future. *Multimed. Tools Appl.* **80**(5), 8091–8126 (2021). <https://doi.org/10.1007/s11042-020-10139-6>

- Kebir, S., Borne, I., Meslati, D.: A genetic algorithm-based approach for automated refactoring of component-based software. *Inf. Softw. Technol.* **88**, 17–36 (2017). <https://doi.org/10.1016/j.infsof.2017.03.009>
- Leyman, P., Van Driessche, N., Vanhoucke, M., De Causmaecker, P.: The impact of solution representations on heuristic net present value optimization in discrete time/cost trade-off project scheduling with multiple cash flow and payment models. *Comp. Oper. Res.* **103**, 184–197 (2019). <https://doi.org/10.1016/j.cor.2018.11.011>
- Li, Z., Kucukkoc, I., Nilakantan, J.M.: Comprehensive review and evaluation of heuristics and meta-heuristics for two-sided assembly line balancing problem. *Comp. Oper. Res.* **84**, 146–161 (2017). <https://doi.org/10.1016/j.cor.2017.03.002>
- Liu, H., Li, G., Ma, Z.Y., Shao, W.Z.: Conflict-aware schedule of software refactorings. *IET Softw.* **2**(5), 446–460 (2008). <https://doi.org/10.1049/iet-sen:20070033>
- Lyu C., Fan Q., Song F., Sinha A., Diao Y., Chen W., et al. Fine-grained modeling and optimization for intelligent resource management in big data processing. *Proc. VLDB Endowment*. (2022) [arXiv: 2207.02026](https://arxiv.org/abs/2207.02026)
- Mahouachi, R., Kessentini, M., Ghedira, K.: A new design defects classification: marrying detection and correction. *Fund. Approaches Softw. Eng.* (2012) p. 455
- Mariani, T., Vergilio, S.R.: A Systematic Review on Search-Based Refactoring. *Inf. Softw. Technol.* **83**, 14–34 (2017). <https://doi.org/10.1016/j.infsof.2016.11.009>
- Masoud, H., Jalili, S.: A clustering-based model for class responsibility assignment problem in object-oriented analysis. *J. Syst. Softw.* **93**, 110–131 (2014). <https://doi.org/10.1016/j.jss.2014.02.053>
- Mkaouer, M.W., Kessentini, M., Ó Cinnéide, M., Deb, K.: Software refactoring under uncertainty: a robust multi-objective approach. In: Conference: Symposium on Search-Based Software Engineering. (2014) <https://doi.org/10.13140/2.1.1894.8485>
- Mokaddem, C.E., Sahraoui, H., Syriani, E.: Recommending model refactoring rules from refactoring examples. In: The 21th ACM/IEEE International Conference. (2018); p. 266. <https://doi.org/10.1145/3239372.3239406>
- Nguyen G., Franco, P.: Mapping high dimensional features onto hilbert curve: applying to fast image retrieval. In: 21st International Conference on Pattern Recognition (ICPR 2012). (2012)
- Ó Cinnéide, M., Boyle, D., Hemati Moghadam, I.: Automated refactoring for testability. In: Proceedings - 4th IEEE International Conference on Software Testing, Verification, and Validation Workshops, ICSTW 2011. (2011) <https://doi.org/10.1109/ICSTW.2011.23>
- O’Keeffe, M., Ó Cinnéide, M.: A Stochastic approach to automated design improvement. In: Conference: Proceedings of the 2nd International Symposium on Principles and Practice of Programming in Java, PPPJ 2003, Kilkenny City, Ireland. (2003); p. 62. <https://doi.org/10.1145/957289.957308>
- O’Keeffe, M., Ó Cinnéide, M.: Getting the most from search-based refactoring. In: GECCO ’07: Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation. p. 1120. (2007) <https://doi.org/10.1145/1276958.1277177>
- O’Keeffe, M., Ó Cinnéide, M.: Towards automated design improvement through combinatorial optimisation. In: International Conference on Software Engineering. (2004)
- O’Keeffe, M., Ó Cinnéide, M.: Search-based software maintenance. *J. Syst. Softw.* **81**, 260 (2006). <https://doi.org/10.1109/CSMR.2006.49>
- Okike, E.: A pedagogical evaluation and discussion about the lack of cohesion in method (LCOM) metric using field experiment. *IJCSI Int. J. Comp. Sci.* **7**(2), 3 (2010)
- Ouni, A., Kessentini, M., Sahraoui, H.: Search-based refactoring using recorded code changes. In: 2013 17th European Conference on Software Maintenance and Reengineering. pp. 221–230. (2013)
- Qayum, F., Heckel, R.: Local search-based refactoring as graph transformation. In: 2009 1st International Symposium on Search Based Software Engineering. pp. 43–46. (2009)
- Qayum, F., Heckel, R.: Search-based refactoring using unfolding of graph transformation systems. *Electron. Commun. EASST*. (2011). <https://doi.org/10.14279/tuj.eceasst.38.577>
- Rapaport, W.: *Philosophy of Computer Science* - Wiley-Blackwell. Wiley-Blackwell, New Jersey (2023)
- Rutenbar, R.A.: Simulated annealing algorithms: an overview. *IEEE Circuits Dev. Mag.* **5**(1), 19–26 (1989). <https://doi.org/10.1109/101.17235>
- Sandhu, P., Singh, H.: A Critical Suggestive evaluation of CK metric. In: 9th Pacific Asia Conference on Information Systems: IT and Value Creation, PACIS 2005. (2005)
- Santos, D., Resende, A., de Castro Lima, E., Freire, A.: Software instability analysis based on afferent and efferent coupling measures. *J. Softw.* **12**, 19–34 (2017). <https://doi.org/10.17706/jsw.12.1.19-34>

- Schotter, E.R., Gerety, C., Rayner, K.: Heuristics and criterion setting during selective encoding in visual decision-making: evidence from eye movements. *Visual Cogn.* **20**(9), 1110–1129 (2012). <https://doi.org/10.1080/13506285.2012.735719>
- Sun, W., Fang, C., Miao, Y., You, Y., Yuan, M., Chen, Y., et al.: Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We? *ACM Transactions on Software Engineering and Methodology*. (2023); [arxiv:2312.00413](https://arxiv.org/abs/2312.00413). [cs]
- Tawosi, V., Jalili, S., Hasheminejad, S.M.H.: Automated software design using ant colony optimization with semantic network support. *J. Syst. Softw.* **109**, 1–17 (2015). <https://doi.org/10.1016/j.jss.2015.06.067>
- Tempero, E., Ralph, P.: A framework for defining coupling metrics. *Sci. Comput. Program.* **166**, 214–230 (2018). <https://doi.org/10.1016/j.scico.2018.02.004>
- van Ginneken, L.P.P.P.: The annealing algorithm revisited. In: *Proceedings of the 2016 on International Symposium on Physical Design. ISPD '16*. New York, NY, USA: Association for Computing Machinery. pp. 109–111. (2016)
- Wongpiang, R., Muenchaisri, P.: Selecting sequence of refactoring techniques usage for code changing using greedy algorithm. In: *2013 IEEE 4th International Conference on Electronics Information and Emergency Communication*. pp. 160–164. (2013)
- Xenos, M., Stavrinoudis, D., Zikouli, K.: Object-oriented metrics a survey. In: *Proceedings of the FESMA 2000, Federation of European Software Measurement Associations*. (2000)
- Zhou, L., Johnson, C.R., Weiskopf, D.: Data-driven space-filling curves. *IEEE Trans. Visual Comput. Graph.* **27**(2), 1591–1600 (2021). <https://doi.org/10.1109/TVCG.2020.3030473>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.